

POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa inżynierska

IOTLAB – PLATFORMA DO ZARZĄDZANIA LABORATORIUM DYDAKTYCZNYM Z SYSTEMEM REZERWACJI I INWENTARYZACJI SPRZĘTU

Krzystof Piechalak, 154065
Jerzy Piersiala, 144328
Kamil Pujanek, 154013
Szymon Szron, 154019

Promotor
dr inż. Ariel Antonowicz

POZNAŃ 2025

Spis treści

Streszczenie **1**

1 Wstęp **2**
1.1 Cel i zakres pracy . . . 2
1.2 Podział pracy . . . 4

2 Zarządzanie laboratorium dydaktycznym **5**
2.1 Historia systemów zarządzania laboratorium . . . 6
2.2 Nowoczesne systemy zarządzania . . . 7
2.2.1 WMS . . . 7
2.2.2 ERP . . . 8
2.2.3 LIMS . . . 8
2.2.4 ELN . . . 8
2.2.5 Inventory Managment Systems . . . 9
2.2.6 Dedykowane systemy . . . 9

3 Przegląd technologii **11**
3.1 C# jako podstawa stosu .NET . . . 11
3.2 Stosy oparte na Pythonie . . . 13
3.3 Java jako fundament stosu Spring . . . 14
3.4 PHP podstawą stosu LAMP . . . 15
3.5 JavaScript jako fundament nowoczesnych stosów webowych . . . 16
3.6 Podsumowanie stosów . . . 19

4 Architektura i model systemu IoTLab **20**
4.1 Strona domowa . . . 20
4.1.1 Panel logowania . . . 21
4.1.2 Slider . . . 22
4.1.3 Notatki . . . 23
4.2 Kokpit użytkownika . . . 24
4.2.1 Moduł Statistics . . . 24
4.2.2 Moduł Objects . . . 25
4.2.3 Moduł Categories . . . 29
4.2.4 Moduł Orders . . . 30
4.2.5 Moduł Users . . . 31
4.2.6 Moduł Profile . . . 32
4.2.7 Moduł Notes . . . 33
4.2.8 Moduł Slider . . . 34

4.2.9 Strony poboczne 34
4.3 Proces wypożyczenia sprzętu 36

5 Implementacja systemu **38**
5.1 Backend 38
5.2 Baza danych 40
5.3 Frontend 41
5.4 Biblioteki dodatkowe 42
5.4.1 Wysyłanie wiadomości e-mail 42
5.4.2 Pages 43

6 Testy jednostkowe i funkcjonalne **44**

7 Podsumowanie i wnioski **46**

Literatura **48**

Streszczenie

Niniejsza praca inżynierska jest poświęcona zaprojektowaniu i implementacji systemu IoTLab – platformy pozwalającej na zarządzenie laboratorium dydaktycznym, z systemem wspomagającym inwentaryzacje oraz możliwością rezerwacji i wypożyczeń sprzętu.

System ten pozwala na znaczące uproszczenie pracy pracownikom naukowo-dydaktycznym, którzy na co dzień opiekują się laboratoriami technicznymi, w których pod ich opieką znajdują się duże ilości różnorodnego sprzętu stanowiącego wyposażenie laboratorium. Platforma IoTLab ułatwia ich pracę dzięki dostarczaniu statystyk, automatyzacji dokumentów, możliwości wprowadzania, przeglądania i śledzeniu sprzętu oraz komunikacji ze studentami. Wdrożone rozwiązanie obejmuje m.in. katalog zasobów, moduł inwentaryzacji, automatyczne powiadomienia e-mail oraz mechanizm rezerwacji z kontrolą administratora. Dodatkowo zaimplementowano panel ogłoszeń do publikowania komunikatów o awariach i pracach serwisowych, a także możliwość dołączania plików PDF oraz materiałów wideo, które będą wspierały proces dydaktyczny. Administrator systemu ma także dostęp do historii rezerwacji co zwiększa transparentność i porządek w zarządzaniu sprzętem.

System ten powstał jako elastyczne narzędzie, które można stosowaćw dowolnym laboratorium dydaktycznym. Pozwala na symulowanie różnego sprzętu i wyposażenia oraz na przechowanie go w jednym miejsce z możliwością zdalnego dostępu. Platforma IoTLab to aplikacja webowa full-stack typu Single-page application (SPA) oparta o stos MERN dzięki czemu jego budowa jak i rozbudowa jest łatwa i szybka. Użytkownicy i studenci łączą się poprzez przeglądarkę internetową z systemem i uzyskują dostęp do zasobów.

Przeprowadzone testy potwierdziły, że IoTLab zwiększa transparentność procesów, ułatwia planowanie zajęć oraz poprawia organizację pracy w laboratorium dydaktycznym. Wskazano także potencjalne kierunki rozwoju systemu, takie jak integracja z kodami QR, systemami uczelnianymi czy aplikacją mobilną.

Summary

This engineering thesis is dedicated to the design and implementation of the IoTLab system – a platform for managing a teaching laboratory, with a system supporting inventory and the ability to reserve and rent equipment. This system significantly simplifies the work of university lecturers who daily manage technical laboratories, which house a large number of diverse laboratory equipment.

The IoTLab platform facilitates their work by providing statistics, document automation, the ability to enter, view, and track equipment, and communication with students. The implemented solution includes a resource catalog, an inventory module, automatic email notifications, and a reservation mechanism with administrator control. Additionally, a notice panel has been implemented for publishing failure and service notifications, as well as the ability to attach PDF files and video materials to support the teaching process.

This system was created as a flexible tool that can be used in any teaching laboratory. It allows for the simulation of various equipment and supplies and their storage in one place with remote access. The IoTLab platform is a full-stack SPA-type web application based on the MERN stack, making its development and expansion quick and easy. Users and students connect to the system via a web browser and gain access to resources.

Tests confirmed that IoTLab increases process transparency, simplifies lesson planning, and improves workflow in the teaching laboratory. Potential directions for system development were also identified, such as integration with QR codes, university systems, and a mobile app.

Rozdział 1

Wstęp

Rozwój technologii cyfrowych w ostatnich dekadach znacząco wpłynął na sposób organizacji pracy w instytucjach edukacyjnych. Uczelnie, podobnie jak przedsiębiorstwa, borykają się z rosnącą liczbą zasobów wymagających odpowiedniego nadzoru oraz udostępniania ich użytkownikom. Jednym z głównych wyzwań jest zarządzanie sprzętem laboratoryjnym, który z jednej strony jest niezbędny do realizacji zajęć dydaktycznych, a z drugiej – stanowi często kosztowną i ograniczoną ilościowo infrastrukturę, która musi być nadzorowana.

Dotychczasowe praktyki w obszarze zarządzania sprzętem i wypożyczalniami doprowadziły do powstania wielu różnych systemów, których zadaniem jest ułatwienie i przyspieszenie pracy osób odpowiedzialnych za wyposażenie i produkty firm oraz organizacji. Chociaż idea systemu zarządzania wywodzi się z gałęzi biznesu, to systemy pozwalające na zarządzanie zasobami stały się również nieodłącznym elementem współczesnych laboratorium sklepów, czy szpitali.

Nie inaczej wygląda to w przypadku laboratoriów dydaktycznych zwłaszcza z zakresu elektroniki i informatyki. W tym przypadku liczba studentów, projektów i zajęć jest bardzo duża, a dostęp do sprzętu musi być czasami planowany z odpowiednim wyprzedzeniem. Brak nowoczesnych narzędzi do zarządzania zasobami wpływa nie tylko na komfort pracy studentów i wykładowców, lecz także na efektywność wykorzystania samego laboratorium oraz sprzętu, który się w nim znajduje.

Często istniejące systemy nie są dostatecznie elastyczne, aby móc stosować je na różnych uczelniach. Powoduje to konieczność opracowania systemu, który dzięki swojej elastyczności, prostocie, przyjaznemu podejściu do użytkownika oraz szybkości pozwoli na łatwą implementację w dowolnym laboratorium, podnosząc przy tym komfort studentów i wykładowców.

System powinien być elastyczny w zakresie prezentowania sprzętu - od najprostszych elementów po skomplikowane urządzenia, pozwolić na łatwe zarządzanie dokumentami, personalizację elementów systemu, wysyłać wiadomości do studentów i wykładowców oraz promować laboratorium dydaktyczne, które obsługuje. Równie ważną cechą systemu musi być jego łatwe utrzymanie i rozwijanie tak, aby laboratorium nie musiało inwestować dużych środków finansowych w posiadanie i obsługę systemu.

1.1 Cel i zakres pracy

Celem niniejszej pracy inżynierskiej jest zaprojektowanie i wdrożenie platformy IoTLab, przeznaczonej do kompleksowego wsparcia zarządzania laboratorium dydaktycznym. System został zaprojektowany w taki sposób, aby zautomatyzować procesy rezerwacji sprzętu oraz inwentaryzacji zasobów, a także zwiększyć przejrzystość dostępnych urządzeń i ułatwić komunikację pomiędzy użytkownikami, a administratorem laboratorium. Dzięki temu rozwiązaniu możliwe jest usprawnie-

nie organizacji zajęć dydaktycznych, efektywniejsze wykorzystanie dostępnej infrastruktury oraz podniesienie komfortu pracy studentów i kadry akademickiej. Zakres pracy obejmował:

1. Przegląd aktualnie dostępnych rozwiązań technologicznych związanych z zarządzaniem laboratoriami dydaktycznymi, systemami rezerwacji oraz inwentaryzacją sprzętu.
2. Opracowanie i realizacja bazy danych przechowującej informacje o sprzęcie laboratoryjnym, jego dostępności, historii wypożyczeń oraz inwentaryzacji.
3. Zaprojektowanie i implementacja systemu rezerwacji sprzętu, obejmującego proces składania wniosków przez użytkowników oraz ich zatwierdzanie przez prowadzących.
4. Opracowanie modułu zgłaszania i zarządzania komunikatami (np. ogłoszenia, awarie, potrzeby serwisowe) dla użytkowników laboratorium.
5. Zaprojektowanie i realizacja serwera odpowiedzialnego za zarządzanie danymi i obsługę żądań użytkowników.
6. Opracowanie i realizacja strony internetowej umożliwiającej przeglądanie zasobów laboratorium, rezerwację sprzętu, dostęp do plików dodatkowych oraz zarządzanie inwentarzem.
7. Weryfikacja i walidacja systemu pod kątem jego funkcjonalności, wydajności oraz użyteczności w środowisku dydaktycznym.

Główne założenia projektowe obejmują implementacje systemu, który będzie intuicyjny, skalowalny, elastyczny pod względem dalszego rozwoju i dostosowania do potrzeb laboratorium dydaktycznego. Praca składa się z 7 rozdziałów, z których każdy przedstawia kluczowe etapy realizacji projektu:

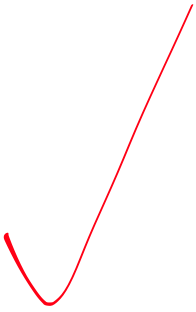
- **Rozdział 1** – zawiera wstęp, określenie celu i zakresu pracy oraz podział zadań pomiędzy członków zespołu;
- **Rozdział 2** – przedstawia przegląd dostępnych rozwiązań technologicznych związanych z zarządzaniem laboratoriami, systemami rezerwacji i inwentaryzacji sprzętu;
- **Rozdział 3** – opisuje zastosowane technologie i narzędzia wykorzystane do implementacji systemu IoTLab;
- **Rozdział 4** – prezentuje projekt systemu, obejmujący wymagania funkcjonalne i niefunkcjonalne, projekt bazy danych, architekturę systemu oraz role użytkowników;
- **Rozdział 5** – poświęcony jest implementacji systemu – zarówno części backendowej, jak i frontendowej – oraz integracji wszystkich modułów w spójne rozwiązanie;
- **Rozdział 6** – zawiera opis testów funkcjonalnych, integracyjnych i manualnych oraz ocenę działania systemu w warunkach dydaktycznych;
- **Rozdział 7** – podsumowuje uzyskane rezultaty, wskazuje najważniejsze wnioski oraz możliwości dalszego rozwoju platformy;

1.2 Podział pracy

W tabeli 1.1 przedstawiono podział zadań zrealizowanych przez poszczególnych członków zespołu.

Tabela 1.1: Podział pracy [opracowanie własne]

Imię i nazwisko	Zadanie
Jerzy Piersiala	1. Przegląd aktualnie dostępnych rozwiązań technologicznych związanych z zarządzaniem laboratoriami dydaktycznymi, systemami rezerwacji oraz inwentaryzacją sprzętu. 5. Zaprojektowanie i realizacja serwera odpowiedzialnego za zarządzanie danymi i obsługę żądań użytkowników. 3. Zaprojektowanie i implementacja systemu rezerwacji sprzętu, obejmującego proces składania wniosków przez użytkowników oraz ich zatwierdzanie przez prowadzących.
Krzysztof Piechalak	2. Opracowanie i realizacja bazy danych przechowującej informacje o sprzęcie laboratoryjnym, jego dostępności, historii wypożyczeń oraz inwentaryzacji. 5. Zaprojektowanie i realizacja serwera odpowiedzialnego za zarządzanie danymi i obsługę żądań użytkowników. 6. Opracowanie i realizacja strony internetowej umożliwiającej przeglądanie zasobów laboratorium, rezerwację sprzętu, dostęp do plików dodatkowych oraz zarządzanie inwentarzem.
Kamil Pujanek	3. Zaprojektowanie i implementacja systemu rezerwacji sprzętu, obejmującego proces składania wniosków przez użytkowników oraz ich zatwierdzanie przez prowadzących. 4. Opracowanie i realizacja strony internetowej umożliwiającej przeglądanie zasobów laboratorium, rezerwację sprzętu, dostęp do plików dodatkowych oraz zarządzanie inwentarzem. 6. Opracowanie i realizacja strony internetowej umożliwiającej przeglądanie zasobów laboratorium, rezerwację sprzętu, dostęp do plików dodatkowych oraz zarządzanie inwentarzem.
Szymon Szron	1. Przegląd aktualnie dostępnych rozwiązań technologicznych związanych z zarządzaniem laboratoriami dydaktycznymi, systemami rezerwacji oraz inwentaryzacją sprzętu. 2. Opracowanie i realizacja bazy danych przechowującej informacje o sprzęcie laboratoryjnym, jego dostępności, historii wypożyczeń oraz inwentaryzacji. 7. Weryfikacja i walidacja systemu pod kątem jego funkcjonalności, wydajności oraz użyteczności w środowisku dydaktycznym.



Rozdział 2

Zarządzanie laboratorium dydaktycznym

Laboratorium dydaktyczne to rodzaj specjalnego pomieszczenia na każdej uczelni. W przeciwieństwie do sal wykładowych, które służą głównie przekazywaniu wiedzy teoretycznej, laboratoria stanowią przestrzeń do rozwijania umiejętności praktycznych. Studenci doskonalą tu zdolności techniczne, uczą się obsługi zaawansowanego sprzętu oraz zdobywają niezbędne kompetencje do wykonywania przyszłych zawodów. Laboratoria dydaktyczne są w tym celu wyposażone w specjalistyczny, często drogi sprzęt oraz w stanowiska pracy przy których studenci zdobywają potrzebne im umiejętności, jednocześnie realizując zadania praktyczne. Współczesne laboratoria pozwalają studentom się w pełni zaangażować, będąc aktywnymi uczestnikami doświadczeń. Sprzyja to lepszemu zapamiętywaniu materiału, rozwijaniu umiejętności analizy oraz rozwiązywania problemów. Praca w laboratorium wspiera również rozwój umiejętności interpersonalnych. Od lepszej komunikacji po współpracę w zespołach i odpowiedzialności za realizowane zadania [8].

Duży wzrost liczby studentów w krajach Ameryki północnej, Europy i Azji wschodniej [58] w ostatniej dekadzie powoduje konieczność optymalizacji dostępu do zasobów uczelnianych i laboratoryjnych. Wraz ze wzrostem liczby studentów jednym z oczywistych rozwiązań jest zwiększenie liczby stanowisk i sprzętu. Nie jest to jednak rozwiązanie idealne, ponieważ większa liczba sprzętu jest zdecydowanie trudniejsza w zarządzaniu i utrzymaniu, a dodatkowo nie da się powiększać liczby stanowisk w nieskończoność ze względu na skończoną powierzchnie dostępną w sali laboratoryjnej. Powoduje to generowanie dodatkowych kosztów.

Warto jednak zwróci uwagę na to, że nie każde ćwiczenie i projekt musi być realizowany w sali laboratoryjnej. Umożliwienie studentom wypożyczania sprzętu tak, aby mogli pracować we własnych mieszkaniach czy akademiku pozwoli rozwiązać problem braku stanowisk dla osób, które chciałyby zostać po zajęciach i pracować nad projektami. Studenci nie będą musieli czekać na zwolnienie się stanowisk albo na zakończenie zajęć innych grup.

Dodatkowo możliwość wypożyczania pozwoli studentom rozwijać swoje pasje i hobby związane z tematyką laboratorium, które nie są objęte programem nauczania [29]. Warto zwrócić uwagę na aspekt użyteczności sprzętu. Możliwość jego wypożyczenia znacząco zwiększa użyteczność, ponieważ sprzęt nie leży bezczynie w przerwach pomiędzy zajęciami, lecz służy zwiększaniu wiedzy i umiejętności studentów nawet po zakończeniu zajęć objętych podstawą programową. Podejście to charakteryzuje się ideą otwartych laboratoriów. Oznacza to, że zasoby są dostępne dla szerszego grona użytkowników i efektywniej wykorzystywane. Dodatkowo system wypożyczeń uczy studentów właściwego obchodzenia się ze sprzętem i pozwala budować świadomość ich wartości. Możliwość wypożyczania zasobów sali dodatkowo pomaga wyrównywać szanse pomiędzy mniej zamożnymi studentami, a tymi bogatszymi, ponieważ biedniejszy student może pożyczyć sprzęt i zrealizować projekt nie wydając pieniędzy, zdobywając przy tym wiedzę i umiejętności, które

pomogą mu na rynku pracy.

Ponadto sprzęt, który znajduje się w laboratorium może nie być łatwo dostępny na rynku z różnych powodów np. przerwania łańcucha dostaw związanych z pandemią. Wypożyczanie go pozwoli na zwiększenie kompetencji i umiejętności studentów nawet w czasie występowania negatywnych zjawisk globalnych.

Jednakże zwiększenie liczby sprzętu oraz umożliwienie wypożyczania studentom niesie za sobą konsekwencje dla opiekuna laboratorium. Musi on mieć kontrolę nad liczbą sprzętu oraz wypożyczeniami. Opiekun potrzebuje świadomości i zawsze aktualnej wiedzy, ile przedmiotów posiada na stanie, które i komu zostały wypożyczone oraz mieć możliwość zbierania statystyk, które pomogą mu zdobyć informacje, jaki sprzęt jest najbardziej potrzebny studentom by móc zwiększysz jego podaż.

Możliwość wypożyczania sprzętu jest dobrym sposobem rozwiązania wielu różnych problemów i niesie za sobą wiele korzyści. Jednakże, aby było to efektywne potrzebne są narzędzia, które pozwolą opiekunowi laboratorium zarządzać wypaczeniami i zasobami sprzętowymi sali dydaktycznej. Narzędzia te muszą odciążać opiekuna laboratorium od części obowiązków związanych z zarządzaniem laboratorium i wspomagać go w inwentaryzacji oraz dostarczać świadomości o dostępnych zasobach.

2.1 Historia systemów zarządzania laboratorium

W dawnych czasach przed nastaniem ery komputerów konieczne było stosowanie papierowej dokumentacji. Wpisywano w niej listę sprzętu będącego na wyposażeniu laboratorium dydaktycznego. Podobnie wyglądała sytuacja z wypożyczaniem sprzętu, wszystkie informacje musiały być spisywane na papierowych dokumentach. Rozwiązanie to posiada oczywiste wady takie jak:

- trudna edytowalność – nie można łatwo zmienić danych na papierowym dokumencie, błędne dane trzeba skreślić, a często trzeba wypełnić nowy ze zmodyfikowanymi danymi;
- łatwość popełnienia pomyłki – dokument nie posiada narzędzi wspomagających wprowadzanie informacji. Łatwo wpisać dane w nieodpowiednim polu co powoduje niepoprawność dokumentu. Negatywne konsekwencje mogą być różne np. błędne statystki wypożyczeń;
- trudne wyszukiwanie informacji – dokumenty muszą być przechowywane w odpowiedniej ustrukturyzowanej formie, aby zapewnić możliwość wyszukiwania. Złe włożenie dokumentu znacząco utrudnia jego znalezienie. A nawet poprawnie umieszczony dokument i tak wymaga przeglądania szuflad i teczek;
- trudność tworzenia kopii zapasowych – chęć utworzenia kopii dokumentów wiąże się z koniecznością ich przepisywania albo kserowania. Jest to czasochłonne i generuje dodatkowe koszty finansowe;
- wysoki koszt papieru – papier w przeliczeniu na ilość przechowywanych informacji jest znacznie droższy w zestawieniu z dyskami komputerowymi. Papierowa dokumentacja nie była narzędziem, które wyręczało opiekuna laboratoriom, była to jednak konieczność, aby posiadał on jakąkolwiek kontrolę nad zasobami.

Pojawienie się komputerów i idącej z nimi cyfryzacji oraz informatyzacji przyniosło możliwość pracy na dokumentach cyfrowych. Było to duże uproszczenie pracy, ponieważ wyszukiwanie stało się łatwiejsze, można było edytować dokument i poprawiać pomyłki bez konieczności przepisywania

dokumentów lub przekreślania błędów. Wykonywanie kopii zapasowych stało się łatwe, a koszty przechowywana dokumentów stały się niższe [33].

Jednakże cyfrowe dokumenty jednie przechowywały informacje w łatwiejsze do utrzymania formie. Nie pozwalały na automatyzacje procesów związanych z wypożyczaniem czy inwentaryzacją. Nie informowały opiekuna o chęci wypożyczenia przedmiotu przez studenta. Nie generowały dokumentów związanych z wypożyczeniami oraz wspomagających inwentaryzacje. Nie zbierały statystyk i nie śledziły automatycznie kto wypożyczył dany przedmiot.

Laboratorium dydaktyczne musi posiadać dedykowany system, który pozwoli nie tylko na łatwe przechowywanie informacji, ale również zautomatyzowanie procesów związanych z wypożyczaniem sprzętu. Dodatkowo powinien mieć mechanizmy wspomagające inwentaryzuję wyposażenia laboratorium.

2.2 Nowoczesne systemy zarządzania

Współcześnie na rynku dostępne są różne systemy, które można wykorzystać do szeroko rozumianego zarządzania laboratorium dydaktycznym. Obejmują one proste aplikacje, które są oparte na kalendarzach rezerwacyjnych czy arkuszach kalkulacyjnych. Wykorzystuje się również zaawansowane platformy umożliwiające kompleksowe zarządzanie przedsiębiorstwem oraz systemy specjalistyczne, które są projektowane z myślą o środowisku akademickim. Rozwiązania te koncentrują się nie tylko na procesie rezerwacji zasobów, ale integrują także dodatkowe moduły, takie jak inwentaryzacja sprzętu, monitorowanie ich stanu technicznego, historię rezerwacji oraz obsługę komunikatów dla użytkowników. Coraz większą rolę odgrywają również systemy webowe i mobilne. Umożliwiają ona studentom oraz pracownikom szybki dostęp do zasobów, niezależnie od miejsca i czasu. Pozwala to na lepsze dostosowanie się do nowoczesnej dydaktyki.

2.2.1 WMS

WMS (Warehouse Management System) to system przeznaczony do zarządzaniami magazynem. Pozwala on na śledzenie przyjętego na stan towaru, jego przemieszczania, ilości i dostępność [52]. Można przy jego pomocy generować raporty i obsługiwać wydania towaru. System ten daje dokładne informacje o tym co dzieje się ze przedmiotami wprowadzonymi do tego systemu oraz pomaga w inwentaryzacji [52]. Istnieją duże i rozbudowane systemy WMS dedykowane dla korporacji oraz małe dla mniejszych firm. Można podjąć się próby obsługi laboratorium dydaktycznego przy pomocy takiego systemu poprzez wprowadzenie do niego wyposażenia laboratorium i wypożyczalnia studentom przedmiotów jako tymczasowe przemieszczanie danych towarów. Niektóre systemy WMS posiadają moduły pozwalające na wypożyczenia.

Jednakże z perspektywy laboratoriów dydaktyczny systemy te są często zbyt drogie, aby móc je wykorzystać do zarządzania laboratorium dydaktycznym. Ponadto są zbyt duże i zaprojektowane z myślą o handlu i logistyce. Większość wbudowanych w nie funkcji nie byłyby wykorzystywane podczas zarządzania laboratorium dydaktycznym. Z tego powodu system WMS nie jest optymalnym rozwiązaniem do wspomagania pracy opiekuna laboratorium.

Implementacja systemu WMS wiąże się też zwykle z wysokimi kosztami licencyjnymi, wdrożeniowymi i utrzymaniowymi. Obejmuje to szkolenie personelu, konfigurację sprzętu oraz wsparcie techniczne [10, 36]. Wiele systemów WMS jest przestarzałych lub stworzone są do dużych operacji, co wiąże się z niską elastycznością i w trudnościami we wdrożeniu w środowisku akademickim, ponieważ jest tam wymagany lekki i dopasowany system [18, 62].

2.2.2 ERP

ERP (Enterprise Resource Planning) to rodzaj systemów zbudowanych w celu zarządzania przedsiębiorstwem [34]. Pozwalają na komunikację z dostawcami, śledzą asortyment przedsiębiorstwa i jego majątek, pozwalają na wystawianie faktur. Można przy ich pomocy sprawować kontrole nad produkcją i magazynem. Umożliwiają wystawianie dokumentów i kontrolę ich obiegu. Wspomagają logistykę [24]. System pomaga w inwentaryzacji i możne zezwalać na wypożyczenia sprzętu. Niektóre systemy ERP mogą mieć konfigurowalne moduły i można zbudować w nich mechanizmy, które mogą pozwalać na wypożyczanie sprzętu studentom.

Przy użyciu takich systemów również można próbować zażądać salą laboratoryjną traktując ją jako firmę, a studentów jako klientów. Jednak implementacja ERP w środowisku akademickim oznaczałaby większe nakłady czasu oraz znacznej adaptacji samego systemu. Moduły finansowo-księgowe lub rozbudowane funkcje produkcyjne byłby zbędne w kontekście dydaktycznym.

Systemy tego typu świetnie sprawdzają się w tym do czego zostały zaprojektowane do zarządzania przedsiębiorstwem, ale po raz kolejny ich wysoka cena oraz niedopasowanie do potrzeb laboratorium sprawia, że nie są one dobrym rozwiązaniem w zarządzaniu salą laboratoryjną. Nadmiar nieużywanych modułów zwiększa koszty utrzymania i komplikuje obsługę, zamiast wspierać procesy rezerwacji sprzętu i prowadzenia inwentaryzacji, które są ważniejsze [24].

2.2.3 LIMS

Laboratory Information Management System (LIMS) to rodzaj systemu służącego za zarządzania laboratorium. Oprogramowanie to pozwala na śledzenie postępu badań i eksperymentów, oraz śledzenie zapasów np. odczynników chemicznych [5]. Pozwalają również na łatwiejsze zarządzanie dokumentacją i na rezerwacje sprzętu laboratoryjnego. Może również umożliwiać wypożyczanie sprzętu przez pracowników laboratorium.

System ten pozwala realizować wymagania jakie są stawiane przed platformą do zarządzania laboratorium dydaktycznym, jednakże jest zaprojektowany z myślą o laboratoriach badawczych, a nie dydaktycznych. Powoduje to, że funkcjonalności takiego systemu nie są odpowiednio dopasowane dla sal laboratoryjnych, nastawionych na naukę studentów. Wiele funkcji, które dostarcza taki system są skrojone pod potrzeby laboratoriów korporacyjnych. Oznacza to, że rozwiązania dostarczane przez LIMS nie są dobrze dopasowane, a cześć z nich jest nadmiarowa i nie będzie wykorzystywana [57].

Istotna wada takich system jest również, to że są płatne. Część z takich systemów korzysta z chmury co czasami jest niezgodne z polityką prywatności. W literaturze zawraca się również uwagę, że wiele systemów LIMS jest zaprojektowanych jako narzędzie do zapewnienia zgodności z regulacjami prawnymi i standardami branżowymi (np. GLP, FDA) co nie ma przełożenia na potrzeby sali dydaktycznej [57].

2.2.4 ELN

ELN (Electronic Lab Notebook) to rodzaj systemu, który dostarcza elektroniczny dziennik laboratoryjny. Pracownicy i studenci mogą zapisywać w nim wyniki badan i eksperymentów. Wyniki mogą być następnie integrowane z innymi narzędziami np. przeznaczonymi do wizualizacji [48].

Systemy te mogą pozwalać na przechowywanie informacji o sprzęcie laboratoryjnym oraz o wypożyczalniach [48]. Niektóre mogą posiadać dedykowane lub personalizowane w zależności od potrzeb moduły dodatkowe, które pozwolą na realizacje zarządzanie sprzętem i wypożyczeniami. W innych wypożyczania będą działać jako kartki przylepiane do tablicy informacyjnej bez wsparcia

w automatyzacji procesów. Należy jednak pamiętać, że są one przeznaczone przede wszystkim do zapisywania wyników eksperymentów. Korzystanie z nich do wypożyczenia nie da pełnej wygody i adekwatnego wsparcia użytkownika.

Systemy ELN zapewniają również funkcje organizacyjne jak możliwość tagowania, automatycznego znacznikowania czasu oraz współpracy w czasie rzeczywistym, co ułatwia tworzenie dokumentacji związanej z eksperymentem [44]. Tak jak w przypadku systemów LIMS, niektóre rozwiązania ELN oferuje przechowywanie danych w chmurze co zwiększa elastyczność, ale często koliduje z polityką uczelni dotyczącej prywatności. Gdy ELN nie jest wykorzystywany bez integracji z LIMS, może to prowadzi do fragmentacji danych i konieczność ręcznego przenoszenia danych między systemami [26]. Obniża to efektywność zarządzania procesami w laboratorium.

2.2.5 Inventory Managment Systems

Innym podejściem mogą być małe systemy do zarządzania asortymentem tak zwane Inventory Managment Systems (IMS). Są to gotowe narzędzia, które pozwalają małym przedsiębiorcą lub osobą prywatnym zarządzać stanem magazynowym małego sklepu. Są one zrealizowane jako platformy internetowe, na których użytkownik zakłada konto. Może wprowadzać dostępny asortyment, śledzić jego liczbę i stan magazynowy [55].

Niektóre pozwalają na integrację magazynu ze sklepem internetowym co można próbować wykorzystać do realizacji modułu wypożyczeń. Zdarza się, że posiadają możliwość wypożyczania przedmiotów z magazynu lub dostarczają konfigurowalne moduły, przy których użyciu można zbudować funkcjonalność wypożyczania sprzętu.

Systemy te, działając często w modelu SaaS (Software as a Service) przechowują dane w chmurze, a dostęp odbywać się może przez przeglądarkę lub aplikację mobilną [68]. Użytkownik dzięki temu uzyskuje możliwość monitorowania zasobów w czasie rzeczywistym, a także integracji z popularnymi platformammi e-commerce oraz systemami płatności [23].

Jednakże po raz kolejny nie jest to odpowiednie rozwiązanie, ponieważ te systemy są przeznaczone dla małych firm i są projektowane z myślą o biznesie, a nie edukacji. Ograniczone są do prostych procesów sprzedażowo-magazynowych. Oznacza to, że brakuje w nich wsparcia dla procesów typowych dla laboratoriów dydaktycznych, takich jak kontrola czasu wypożyczeń czy obsługa wielu ról użytkowników (administrator, student, wykładowca) [56]. Taka adaptacja wymagałaby licznych modyfikacji i obejść, które dodatkowo obniżają użyteczność w środowisku akademickim. Ponadto platformy te są płatne, a w planie darmowym pozwalają np. na ograniczoną liczbę przedmiotów albo posiadają limit zamówień na miesiąc.

2.2.6 Dedykowane systemy

Rozwiązania korporacyjnie lub biznesowe nie sprawdzają się najlepiej w zarządzaniu laboratorium dydaktycznym. Z tego powodu konieczne jest się zbudowanie systemu dedykowanego zarządzaniu laboratoriom dydaktycznym.

Systemy takie istnieją i posiadają kluczowe funkcjonalności umożliwiające zarządzanie laboratorium dydaktycznym. Systemy te są budowane przez pracowników lub opiekunów laboratorium. Dzięki temu zbudowanie systemu jest tańsze i lepiej dopasowane do potrzeb.

Jednym z dedykowanych systemów jest nigeryjski system zarządzania laboratorium [1]. Pozwala on na logowanie użytkowników, wprowadzanie i modyfikacje przedmiotów znajdujących się w sali laboratoryjnej. Pamięta historię wyposzczeń, umożliwia rezerwacje oraz dostarcza statystyk. Innym wartym wspomnienia dedykowanym systemem jest chiński system służący do zarządzania wyposażeniem sali laboratoryjnej [65].

Na podstawie przeglądu dostępnych rozwiązań zauważyć można, że systemu do zarządzania laboratorium dydaktycznym powinny:

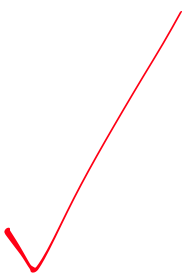
- umożliwiać dodawanie i przeglądnie dostępnego sprzętu;
- umożliwiać zbieranie statystyk;
- pozwalać na zdalny dostęp do zasobów laboratoryjnych;
- wykorzystywać darmowe narzędzia: dzięki temu można zbudować go bez wydawania pieniędzy, a utrzymanie nie wymaga kupowania licencji dzięki czemu fundusze można inwestować w sprzęt zamiast rachunków za licencje;
- pozwalać na utworzenie kont użytkowników tak aby posiadać kontrolę nad tym kto ma jaki dostęp do danych zasobów;
- posiadać tabelaryczny widok przedmiotów;
- umożliwiać dostęp do systemu poprzez urządzenia mobilne.

System powinien posiadać dodatkowe funkcjonaliści takie jak:

- ułatwiać inwentaryzuje dzięki generowaniu pomocnych dokumentów takich jak lista przedmiotów danej kategorii lub znajdujących się w danej sali.;
- umożliwiać komunikację pomiędzy opiekunami laboratorium, a studentami;
- system sam generuje dokumenty np. potrzebne przy wypożyczeniu.

Warto jednak zwrócić uwagę na istotny problem. Wspomniane systemy są zbudowane w technologiach trudnych do nauki lub wymagających znajomości kilku technologii. Ponadto systemy te są mało elastyczne, ponieważ postały w konkretnych celach i realizują konkretne zadania w danym laboratorium. Próba przeniesienia ich do laboratoriów innych uczelni może nie być łatwa. Dodatkowo różnica miedzy różnymi laboratoriami mogą nie pozwali na dopasowanie takiego systemu do innej sali.

Konkluzja jest następująca, istniejące systemy do zarządzania laboratoriami są bardzo dobre i świetnie realizują swoje zadania w laboratoriach, dla których zostały zbudowane, ale nie są dostatecznie elastyczne by przenosić je do innych uczelni. Konieczne jest zbudowanie systemu, który będzie łatwiej wpasowywał się w różne niuanse w laboratoriach dydaktycznych.



Rozdział 3

Przegląd technologii

Aplikacje webowe stają się coraz bardziej złożone i wymagają różnych elementów takich jak bazy danych, integracje z API (Application Programming Interface), systemy autoryzacji i bezpieczeństwa, responsywnego interfejsu graficznego, dynamicznych komponentów czy też systemu przechowywania plików. Z tego względu tworzenie nowoczesnych aplikacji i stron wymaga połączenia wielu narzędzi: języków programowania, frameworków, bibliotek oraz baz danych.

Niektóre narzędzia współpracują ze sobą lepiej, a inne gorzej. Z tego powodu tworzy się dobrane pakiety narzędzi, które ułatwiają budowę i utrzymanie aplikacji. Spójność narzędzi powoduje mniej problemów z kompatybilnością. Zdarza się, że programista uczy się i wykorzystuje tylko jedną technologię w wielu miejscach (np. język JavaScript zarówno na frontendzie jak i backendzie) co przyśpiesza pracę i umożliwia standaryzację w projekcie. Spójność technologii ma również znaczenie z perspektywy zarządzania zespołem programistów.

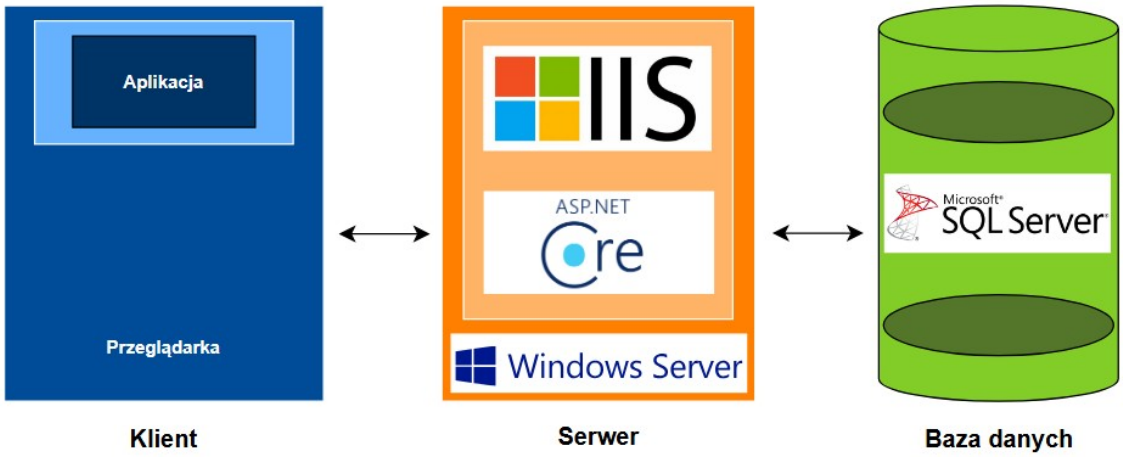
Takie połączenie konkretnych technologii i narzędzi w jeden zestaw nazywamy stosem technologicznym, który umożliwia budowę aplikacji. Wybór odpowiedniego stosu określa zakres funkcjonalności tworzonej aplikacji oraz poziom jej personalizacji [46]. W pracy [22] autorzy zauważają również, że wybór stosu technologicznego, ma istotne znaczenie dla architektury, kosztów, łatwości nauki oraz wsparcia społeczności. Wpływa także na skalowność, wydajność i efektywność procesu tworzenia aplikacji webowych. Klasyczne stosy technologiczne obejmują frontend, backend, serwer, bazę danych, system operacyjny oraz przeglądarkę, dzięki której możliwa jest interakcja z użytkownikiem aplikacji [46]. Jedną z głównych zalet wynikających z korzystania ze stosu jest skrócenie czasu tworzenia aplikacji oraz obniżenie kosztów jej utrzymania, dzięki czemu cały proces jest bardziej efektywny [46]. Dodatkowo autor pracy [46] zauważa, że programiści, który potrafią pracować z pełnym stosem są szczególnie cenieni na rynku pracy, gdyż potrafią samodzielnie projektować i utrzymywać aplikację od frontendu po backend. Dobrze dobrany stos ułatwia wdrażanie nowych członków do projektu oraz przekłada się na ogólną wydajność całego zespołu [22]. Na przestrzeni lat, powstały pewne szczególnie popularne zestawy technologii, które zdobyły już uznanie w środowisku programistów, ze względu na swoją stabilność, szerokie zastosowanie i dobre wsparcie społeczności.

3.1 C# jako podstawa stosu .NET

Język C# charakteryzował się silnym typowaniem, składnią inspirowaną językami C i Java, a także rozbudowanym systemem blibliotek. Jedynym z najważniejszych zalet języka C# jest jego wydajność i bezpieczeństwo. Jest on kompilowany do CIL (Common Intermediate Language) oraz uruchamiany w środowisku .NET CLR (Common Language Runtime). Dzięki temu łączy

zalety języków wysokopoziomowych z optymalizacją wydajnościową [35]. Od czasu wprowadzenia .NET Core, stał się też wieloplatformowy, przez co może działać na systemach Windows, Linux oraz macOS, a to zwiększa jego uniwersalność [25]. Obsługuje on też programowanie asynchroniczne (async/await), programowanie generyczne oraz elementy programowania funkcyjnego, co sprawia, że jest narzędziem wykorzystywanym w wielu dziedzinach, takich jak aplikacje webowe, desktopowe, mobilne, gry komputerowe czy systemy korporacyjne [16]. Jest on szeroko stosowany w sektorach związanych z finansami, administracją publiczną oraz opieką zdrowotną, czyli tam, gdzie wymagane są wysokie standardy bezpieczeństwa i stabilności systemów.

Stos .NET należy do jednych z najczęściej stosowanych środowisk do rozwoju oprogramowania korporacyjnego. Jak widać na rys. 3.1 poza wykorzystywaniem języka C#, jego częścią jest również framework ASP.NET Core, serwery aplikacyjne IIS (Internet Information Services) lub Kestrel. Działa zwykle na Windows Server. Baza danych zwykle jest oparta o SQL Server. Umożliwia to budowę nowoczesnych aplikacji webowych i usług chmurowych. Najważniejszymi zaletami korzystania z tego stosu jest jego wysoka wydajność i silne wsparcie dla bezpieczeństwa. ASP.NET Core korzysta z kompliacji just-in-time (JIT) oraz zoptymalizowanych bibliotek systemowych, co przekłada się na szybkie przetwarzanie żądań HTTP i niski narzut na zasoby serwera [25]. Posiada też wbudowane mechanizmy autoryzacji i uwierzytelnienia m.in. Identity, JWT oraz integracje z Active Directory. Są one szczególnie istotne w aplikacjach korporacyjnych. Integracja z ekosystemem Microsoft ułatwia korzystanie z narzędzi takich jak Visual Studio, Azure DevOps, Active Directory czy MS SQL Server. Sprawdza się także do obsługi dużej liczy użytkowników [35]. Platforma .NET jest rozwijana od ponad dwóch dekad, a programiści C# należą do jednej z największych grup zawodowych w IT [14].



Rysunek 3.1: Stos oparty o .NET [opracowanie własne]

Mimo wielu zalet ma on też pewne ograniczenia. Chociaż ASP.NET Core jest rozwiązaniem typu open-source, w praktyce wiele wdrożeń środowiskowych korzysta z płatnych rozwiązań i usług. Są to np. licencje na SQL Server lub środowiska chmurowe Azure. Może to stanowić barierę dla projektów, których celem jest szybkie i niskokosztowe wdrożenie [25]. Wdrożenie pełnego stosu .NET jest też złożone i wymaga konfiguracji wielu elementów, co podnosi próg wejścia.

Stos .NET został zaprojektowany pod duże organizacje, co jest zaleta w przypadku złożonych systemów. Dla mniejszych projektów oznacza to konieczność pracy w skomplikowanym środowisku i skutkuje nadmiernym narzutem technicznym i organizacyjnym, co utrudniając sprawne wdrożenie.

3.2 Stosy oparte na Pythonie

Python został zaprojektowany na początku lat 90. przez Guido van Rossuma jako prosty, zrozumiały i czytelny język. Oparty jest na zasadzie „There should be one – and preferably only one – obvious way to do it" [59]. Należy też do jednych z najbardziej rozpowszechnionych języków programowania współczesnej informatyki. Czytelność kodu oraz niska bariera wejścia były jego głównym celem od początku powstania. Miało to ułatwić naukę programowania i przyśpieszyć tworzenie oprogramowania.

Choć na początku język ten należał do niszowego języka akademickiego, z czasem stał się jednym z najczęściej używanych na świecie [49]. Wynika to z jego uniwersalności. Python jest stosowany zarówno w aplikacjach webowych, jak i w analizie statystycznej, uczeniu maszynowym, automatyzacji procesów, czy też edukacji informatycznej.

Do jego największych zalet należy wspomniana już łatwość nauki i czytelność składni. Kod przypomina język naturalny, dzięki czemu jest intuicyjny i łatwiejszy w nauce niż w przypadku innych języków. Sprawia to, że często jest wybierany jako pierwszy język nauczania programowania [28]. Znajduje on też zastosowanie w wielu dziedzinach, dzięki czemu programista może korzystać z jednego języka, w wielu różnych kontekstach. Posiada również rozbudowany ekosystem bibliotek i frameworków, w tym NumPy i Pandas do analizy danych, TensorFlow i PyTorch do uczenia maszynowego, czy też Selenium do automatyzacji testów Aplikacje napisane w Pythonie mogą działać na wielu systemach operacyjnych (Windows, Linux, macOS), a dodatkowo wspiera on programowanie obiektowe, funkcyjne i proceduralne [41, 59].

Pierwotnie nie był on językiem stworzonym do budowy aplikacji webowych, jednak jego prostota i możliwości rozbudowy zaowocowały powstaniem frameworków takich jak Django i Flask. To sprawiło, że język ten stał się istoty w dziedzinie web developmentu, zaraz obok języków takich jak Java czy JavaScript.

Django często bywa określane frameworkiem full-stack, ponieważ oprócz warstwy logiki oferuje również system szablonów do tworzenia interfejsu użytkownika. Odznacza się podejściem „batteries-included" co oznacza, że w ramach tego narzędzia, programista otrzymuje kompletne środowisko do budowy aplikacji. Typowy stos obejmuje wspomniany wcześniej język Python jako warstwy logiki aplikacyjnej. Wykorzystuje się również relacyjną bazę danych (zwykle PostgreSQL lub MySQL do mniejszych aplikacji). Serwerem aplikacyjnym jest Gunicorn lub uWSGI, a serwerem HTTP: Nginx lub Apatche, działający jako reverse proxy. Sam framework pełni rolę zaplecza (backend), odpowiadając za logikę, przetwarzanie żądań i komunikację z bazą danych. Django udostępnia własny system ORM (Object-Relational Mapping), panel administracyjny generowany automatycznie, mechanizmy autoryzacji użytkowników oraz system szablonów HTML. Dzięki wbudowanym zabezpieczeniom chroni przed najczęstszymi atakami typu SQL Injection, XSS, czy też CSRF. Umożliwia również tworzenie zaawansowanych aplikacji z niewielką ilością zewnętrznych zależności [17].

Inne podejście reprezentuje Flask. Jest on mikroframeworkiem. Oznacza to, że sam w sobie oferuje jedynie minimalny zestaw funkcji, jak routing czy obsługę żądań HTTP (oparta na bibliotece Werkzeug). Najczęściej do stosu wybiera się również SQLAlchemy jako ORM, JINja2 jako system szablonów. Podobnie jak Django, stos Flask obsługuje relacyjne bazy danych (PostgreSQL, MySQL, SQLite) oraz serwery aplikacyjne Gunicorn lub uWSG w połączeniu z serwerem HTTP - Nginx. Programista sam musi dobrać i zintegrowować potrzebne komponenty w celu zapewnienia autoryzacji oraz obsługi panelu administracyjnego (np. Flask-Login, Flask-Admin)[15].

Pomimo, że oba frameworki służą do budowy aplikacji webowych i są stosunkowo popularne, ich filozofia projektowa jest różna. Django stawia na kompletność oraz narzuca programiście okre-

śloną strukturę pracy w architekturze MTV (Model-Template-View). Jest to szczególnie przydatne w dużych projektach, gdzie standaryzacja kodu oraz praca w większych zespołach jest szczególnie ważna. Flask oferuje dużą elastyczność oraz swobodę, szczególnie w wyborze architektury, bo nie narzuca konkretnego wzorca. Sprawia to, że Flask jest szczególnie przydatny przy prototypowaniu i budowanie prostych usług webowych i mikroserwisów [12].

Dzięki licznym narzędziom wbudowanym w framework, wysokiemu bezpieczeństwu oraz uporządkowanej strukturze Django oferuje szybkie tworzenie kompletnych aplikacji. Sprawdza się szczególnie dobrze w dużych aplikacjach i serwisach typu e-commerce czy systemach edukacyjnych. Natomiast Flask wyróżnia się prostotą i lekkością. Jego elastyczność sprawia, że jest dobrym narzędziem wykorzystywanym do budowy API oraz prototypów aplikacji [12, 15].

3.3 Java jako fundament stosu Spring

Java należy do najdłużej utrzymujących się nowoczesnych języków programowania wysokiego poziomu. Język ten został zaprojektowany w 1995 roku przez zespół pod kierownictwem Jamesa Goslinga w firmie Sun Microsystems, a obecnie rozwijany jest przez Oracle. Od samego początku Java wyróżniała się podejściem „write once, run anywhere". Napisany kod mógł być uruchamiany na dowolnym systemie obsługującym maszynę wirtualną Javy (JVM). Koncepcja ta pozwoliła na znacznie uniezależnienie aplikacji od konkretnej platformy sprzętowej i systemowej [13].

Java przez lata ewoluowała i stała się podstawą dla całego ekosystemu narzędzi i frameworków, które ułatiwają budowę prostych aplikacji oraz złożonych systemów. Dzięki swojej uniwersalności znalazła zastosowanie w aplikacjach webowych, systemach wbudowanych oraz w oprogramowaniu mobilnym. Java jest też językiem bazowym dla platformy Android, co dodatkowo zwiększyło jego popularność [40].

Zaletą tego języka jest jego wieloplatformowość zapewniana przez JVM. Jego stabilność i niezawodność spowodowały, że stał się standardem w systemach wymagających wysokiej jakości i bezpieczeństwa, takich jak bankowość czy administracja publiczna. Od ponad dwóch dekad rozwijane są liczne frameworki i narzędzie wspierające programistów. Zaowocowało to dużym ekosystemem oraz społecznością skupioną wokół Javy.

Jednym z najbardziej znaczących frameworków jest Spring Framework. Jego rozszerzeniem i uproszczoną wersją stał się Spring Boot. Jest on obecnie jednym z najczęściej wybieranych stosów technologicznych do budowy aplikacji. Został on opracowany, aby uprościć tworzenie i konfigurację aplikacji webowych, które w klasycznym Springu były skomplikowane i wymagały dużej ilości konfiguracji XML. W Spring Boot, mechanizm automatycznej konfiguracji oraz zestaw starterów, umożliwia uruchomienie kompletnej aplikacji webowej przy niskim nakładzie pracy [6].

Typowy stos obejmuje wbudowany serwer aplikacyjny TomCat (lub Jetty), relacyjną bazę danych PostgreSQL (lub MySQL, albo Oracle), warstwę frontendową z integracją z szablonami Thymeleaf oraz JSP, lub zewnętrznymi frameworkami jak React, Angular lub Vue. Dodatkowo ma wiele narzędzi wspierających dostęp do danych (Spring Data), uwierzytelnienie (Spring Security) czy też mikroserwisy i chmurę Spring Cloud). Sam framework, działając na warstwie backendowej, oferuje mechanizmy REST API, integrację z bazami danych oraz zarządzanie zależnościami.

Do jego głównych zalet, należy wspomnieć zredukowaną liczbę plików konfiguracyjnych. Dzięki automatycznej konfiguracji i predefiniowanym starterom np. @SpringBootApplication, łączy trzy ważne adnotacje w jednej [7]. Wbudowany serwer eliminuje konieczność zewnętrznej konfiguracji, a narzędzia takie jak Actuator zapewniają monitoring i metryki, przydatne przy pracy aplikacji [45]. Dlatego, że framework wspiera też natywnie architekturę mikroserwisową, to jest idealny do nowoczesnych i skalowalnych rozwiązań [63].

Mimo zalet, stos ten wiąże się z ograniczeniami, które w kontekście tego systemu okazują się problemem. Tworzenie aplikacji korzystając ze Spring Boot oraz typowego stosu technologicznego jest wolniejsze niż w innych stosach. Aplikacje wymagają stworzenia pełnej struktury projektu, a krzywa uczenia się Javy oraz wspominanego frameworka jest stosunkowo wysoka w porównaniu do tych opartych na JavaScripcie. Powoduje to że zespół programistów potrzebuje więcej czasu na przygotowanie działającego prototypu. Dodatkowo Spring Boot działa wyłącznie na warstwie backendowej, co powoduje, że interfejs użytkownika musi być zbudowany w innym środowisku, np. przy użyciu Reacta lub Angulara. Powoduje to konieczność integracji dwóch odrębnych ekosystemów technologicznych co zwiększa złożoność całego projektu [50]. Spring Boot został zaprojektowany głównie z myślą o systemach klasy enterprise, czyli tam gdzie wymaga się skalowności, wysokiej dostępności i rozbudowanych mechanizmów bezpieczeństwa. W przypadku niewielkiego systemu, który z czasem może być rozbudowywany, funkcje te wydają się zbędne, a ich implementacja generowałaby dodatkowy narzut czasu, który nie jest współmierny do efektów [7].

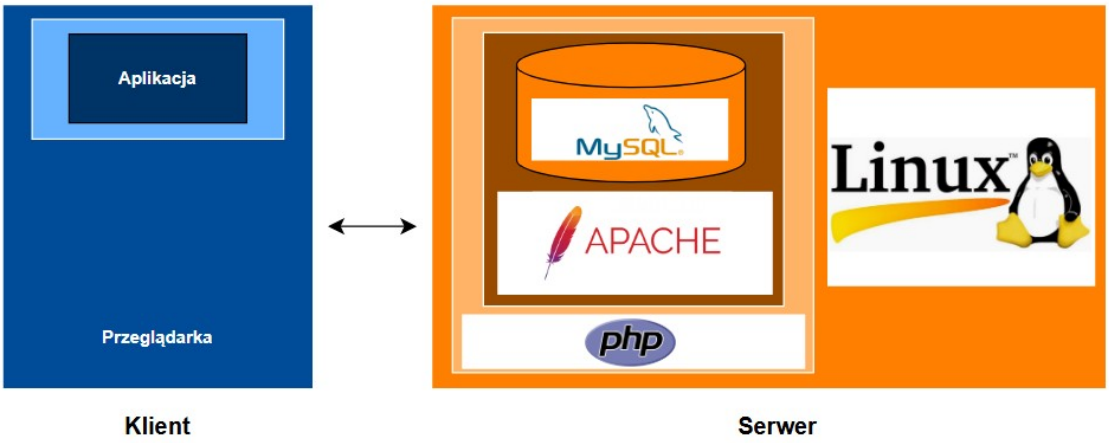
3.4 PHP podstawą stosu LAMP

Język skryptowy PHP należy do jednych z najstarszych i najbardziej rozpowszechnionych. Wykorzystywany jest głównie w programowaniu aplikacji webowych. Opracowany został w 1994 roku przez Rasmusa Lerdorfa. Choć na początku służył do prostego generowania dynamicznych stron internetowych, z czasem ewoluował w pełnoprawny język programowania zorientowany na aplikacje internetowe. Charakteryzuje się łatwą integracją z HTML oraz bazami danych - przede wszystkim MySQL [64].

Składnia PHP jest stosunkowo łatwa do nauczenia, co sprawia, że język ten jest popularnym wyborem do nauki wśród początkujących programistów [66]. Umożliwia również bezpośrednie osadzanie skryptów w kodzie HTML, co ułatwia tworzenie dynamicznych stron. PHP przez lata stanowił fundament wielu systemów CMS (np. WordPress, Drupal, Joomla) oraz dysponuje on rozbudowanym ekosystemem bibliotek, i frameworków. Dzięki szerokiemu wsparciu ze strony dostawców hostingu, jest często specjalnie zoptymalizowany pod kątem aplikacji PHP. To właśnie powszechne wykorzystanie PHP w połączeniu z łatwością integracji z serwerami i relacyjnymi bazami danych doprowadziło do powstania jednego z najbardziej znanych stosów technologicznych.

Dominującym rozwiązaniem w rozwoju aplikacji webowych przez wiele lat był stos LAMP. Jak widać na rys. 3.2, jest on zbiorem oprogramowania typu open-source, a tworzy go:

- Linux - system operacyjny, na którym działa serwer;
- Apache - serwer WWW, który obsługuje żądania HTTP;
- MySQL - system zarządzania relacyjną bazą danych;
- PHP - język skryptowy używany do budowy logiki aplikacji po stronie serwera.



Rysunek 3.2: Stos LAMP [opracowanie własne]

Był szczególnie popularnym wyborem dla prostych serwisów internatowych i systemów zarządzania treścią (CMS), ponieważ był prosty we wdrożeniu, miał niskie koszty utrzymania oraz szerokie wsparcie społeczności. LAMP był jednym z pierwszych stosów korzystających z otwartego oprogramowania [19] i stanowił fundament klasycznych architektów webowych, zapewniając stabilność i niezawodność.

Ograniczenia tego stosu stały się coraz bardziej widoczne wraz z dynamicznym rozwojem technologii, zmieniającymi się oczekiwaniami użytkowników, rosnącymi wolumenami danych wykorzystywanych w aplikacjach webowych, czy potrzebą interaktywnego interfejsu. Radzi sobie gorzej z dużym obciążeniem i dużą liczbą jednoczesnych połączeń, ponieważ PHP nie działa synchronicznie. Stanowi to wąskie gardło w aplikacjach wymagających obsługi w czasie rzeczywistym [3] i może być zauważalnie wolniejsze od stosów, które korzystają z asynchronicznego modelu I/O.

LAMP jest również mniej elastyczny w budowie nowoczesnych i dynamicznych aplikacji. Stos ten jest częściej kojarzony z prostymi, statycznymi serwisami internetowymi, podczas gdy MERN jest lepiej dostosowany do aplikacji wymagających dynamicznego interfejsu użytkownika i aktualizacji danych [43].

Realizacja aplikacji w LAMP wymaga znajomości co najmniej trzech różnych języków programowania i zapytań (PHP, SQL i JavaScript). Oznacza to, że utrzymanie i rozwój projektu może być trudniejsza, w porównaniu ze stosami opierającymi się na jednym lub dwóch językach programowania. LAMP wymaga dodatkowego nakładu czasu na naukę i pracę w różnych językach, co może stanowić barierę, szczególnie dla mniej doświadczonych programistów lub zespołów [43].

Warto też wspomnieć, że MySQL jest relacyjną bazą danych, która dobrze działa przy uporządkowanych danych, ale gorzej radzi sobie z rosnącą liczbą złożonych, nieustrukturyzowanych danych [11].

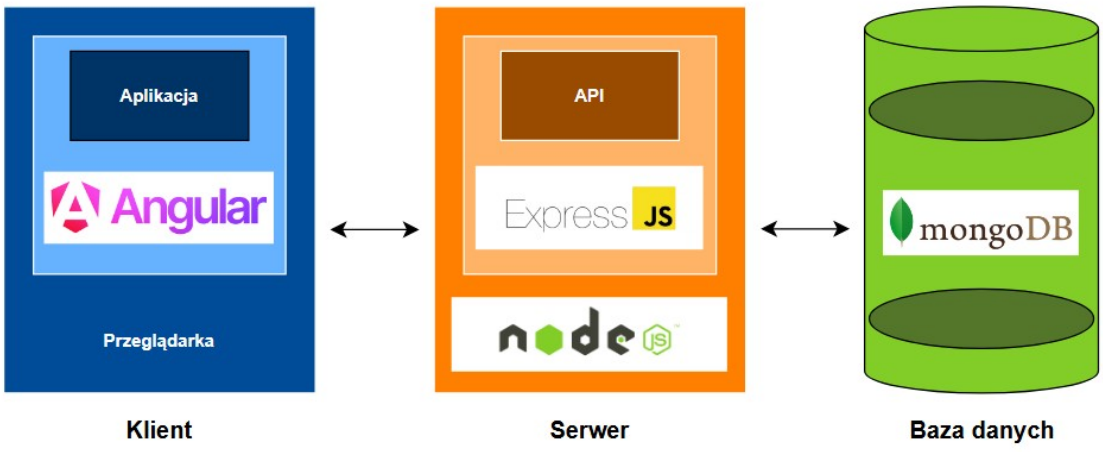
3.5 JavaScript jako fundament nowoczesnych stosów webowych

JavaScript to jeden z najważniejszych języków programowania. Został stworzony w 1995 roku przez Brendana Eicha w firmie Netscape Communications jako język skryptowy. Był przeznaczony do dynamicznego wzbogacania stron internetowych. Początkowo pełnił zwykle rolę języka klienckiego. Pozwalał na walidację formularzy i podstawowe interakcje w przeglądarce. Wraz z rozwojem silników JavaScript oraz środowiska uruchomieniowego Node.js, jego rola wzrosła i dziś jest to język pełnostosowy, który jest używany zarówno po stronie klienta, jak i serwera [9].

Jego najważniejszą zaletą jest uniwersalność. Język JavaScript można wykorzystać do budowy kompletnych aplikacji webowych w modelu klient-serwer. Zapewnia to spójność procesu wytwarzania oprogramowania, gdyż zarówno frontend (np. React, Angular, Vue), jak i backend (Node.js, Express) mogą być pisane w tym samym języku [30]. Posiada też asynchroniczny model obsługi zdarzeń i nieblokujące I/O. Umożliwia to budowę aplikacji, która obsługuje dużą liczbę jednoczesnych połączeń, bez obciążania serwera [54]. Stał się on dzięki temu podstawą systemów czasu rzeczywistego, jak komunikatory, systemy streamingowe czy aplikacje IoT. JavaScript odznacza się też rozbudowanym ekosystemem bibliotek i frameworków. Jest to też najczęściej używany język programowania na świecie [49]. Zgodnie z badaniami, rejestr NPM (Node Package Manager - otwarte repozytorium oprogramowania) zawiera ponad 2 miliony pakietów, a inne analizy sięgają nawet 3 milionów. Dzięki temu, JavaScript jest dzisiaj fundamentem nowoczesnych stosów technologicznych, takich jak MERN oraz MEAN, które są jednymi z najczęściej wykorzystywanych stosów przy budowanie aplikacji webowych. Każdy z nich umożliwia tworzenie w pełni działających aplikacji webowych. Powstanie i upowszechnienie się wcześniej wskazanych stosów było według [22] odpowiedzią na rosnące wymagania rynku i konieczności zapewnienia sprawdzonych środowisk do budowy aplikacji. Dobór odpowiedniego stosu może mieć zatem strategiczne znaczenie dla sukcesu projektu.

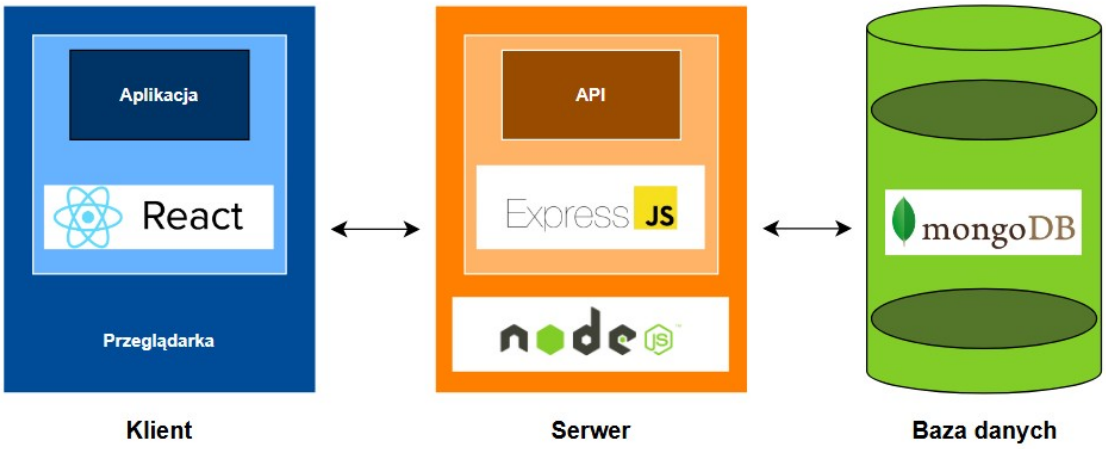
Rosnąca złożoność aplikacji internetowych, potrzeba obsługi dynamicznych interfejsów i dużej liczby użytkowników jednocześnie spowodowały, że klasyczne podejścia, takie jak stos LAMP zaczęły ustępować bardziej nowoczesnym rozwiązaniom. Odpowiedzią na to były stosy oparte w całości na języku JavaScript. Jednym z nich jest stos MEAN. Jak widać na rys. 3.3 opiera się on na technologii:

- MongoDB – nierelacyjna baza danych, przechowująca dane w formacie dokumentów JSON, co ułatwia obsługę dynamicznych i nieustrukturyzowanych danych;
- Express.js – lekki framework backendowy dla Node.js, który upraszcza budowę API i logiki serwera;
- Angular – framework frontendowy rozwijany przez Google, zapewniający strukturalne podejście do tworzenia złożonych aplikacji jednostronicowych;
- Node.js – środowisko uruchomieniowe JavaScript po stronie serwera, umożliwiające asynchroniczne i wydajne przetwarzanie wielu żądań jednocześnie.



Rysunek 3.3: Stos MEAN [opracowanie własne]

Kolejnym stosem wykorzystującym język JavaScript, jest MERN. Jak pokazuje rys. 3.4 jedyną różnicą jest wykorzystanie biblioteki frontendowej React, zamiast frameworka Angular. React został opracowany przez Facebooka i umożliwia budowę dynamicznych, komponentowych interfejsów użytkownika,



Rysunek 3.4: Stos MERN [opracowanie własne]

Wspólną cechą obu stosów jest także rozbudowane wsparcie społeczności i zbiór narzędzi, które przyspieszają proces wytwarzania oprogramowania. Ich popularność sprawiła, że dostępna jest duża liczba gotowych bibliotek, komponentów i materiałów edukacyjnych, co obniża próg wejścia i zwiększa produktywność zespołów [2].

Pomimo licznych podobieństw, kluczowa różnica tkwi w warstwie frontendowej. MEAN wykorzystuje Angular, czyli rozbudowany framework rozwijany przez Google. Narzuca on ściślejszą strukturę kodu oraz model architektoniczny typu MVC (Model–View–Controller). Powoduje to, że aplikacje wykorzystujące Angulara cechują się wysoką standaryzacją co jest korzystne szczególnie przy złożonych projektach korporacyjnych. Zespoły są tam liczne i wymagają spójnego podejścia do implementacji. Dodatkową cechą jest wyższa niż w przypadku MERN, krzywa uczenia się i mniejsza elastyczność przy tworzeniu prototypów [38, 53].

Stos MERN opiera się na React. Jest to biblioteka rozwijana przez Facebook (Meta). W przeciwieństwie do Angulara nie narzuca sztywnej architektury aplikacji. React skupia się głównie na warstwie widoku (View). Umożliwia to budowę dynamicznych interfejsów użytkownika z wykorzystaniem podejścia komponentowego. Sprawia to, że MERN jest bardziej intuicyjny i łatwiejszy do nauki. Pozwala również na szybkie prototypowanie i stopniowe rozbudowywanie aplikacji o kolejne moduły [27, 47]. Dzięki temu MERN sprawdzi się lepiej w mniejszych projektach, takich jak systemy dydaktyczne lub aplikacje MVP (Minimum Viable Product), które z czasem mogą rozwijać się w bardziej złożone rozwiązania.

Właśnie te cechy sprawiają, że MERN jest szczególnie odpowiedni dla budowanego systemu. Zapewnia on odpowiednią spójność środowiska, ponieważ wykorzystuje jeden język - JavaScript, niski próg wejścia dla zespołu, szybkie prototypowanie i intuicyjny rozwój interfejsu użytkownika. MERN pozwala również integrować logikę serwera z dynamicznym frontendem, co jest bardzo ważne dla systemu zarządzania i wypożyczenia sprzętu laboratoryjnego.

3.6 Podsumowanie stosów

Pomimo, że stos .NET jest rozwiązaniem bardzo dojrzałym i wydajnym, które sprawdza się najlepiej w dużych aplikacjach korporacyjnych wymagających wysokiego poziomu bezpieczeństwa, integracji i skalowalności, to w kontekście niniejszego projektu jest zbyt kosztowny i nadmiarowo skomplikowany [60].

Dla stosów opartych na Pythonie, takich jak Django i Flask, również można zauważyć pewne ograniczenia. Do najważniejszych należy spójności technologicznej, bo oba frameworki pełnią głównie rolę backendu, przez co dynamiczny i interaktywny frontend musiałby zostać zaimplementowany w osobnych technologiach, takich jak React czy Angular. Powoduje to konieczność łączenia różnych ekosystemów, co zwiększa złożoność projektu. Choć Django jest bogate w funkcje, dostarcza wiele elementów nadmiarowych dla naszego systemu (m.in. rozbudowany panel administracyjny i skomplikowany ORM), a Flask wymaga od programisty samodzielnej integracji wielu bibliotek, co podnosi koszty utrzymania i spowalnia prototypowanie.

Podobne trudności wiążą się ze stosem Spring Boot opartym na Javie. Ten framework jest niezwykle popularny w sektorze enterprise, czyli tam gdzie liczy się przede wszystkim stabilność, bezpieczeństwo i skalowalność. W kontekście niewielkiego projektu dydaktycznego, Spring Boot okazuje się rozwiązaniem zbyt ciężkim i czasochłonnym we wdrożeniu. Złożony ekosystem, długa krzywa uczenia się oraz konieczność łączenia backendu w Javie z frontendem w JavaScripcie generują dodatkowy narzut technologiczny, który tak jak w przypadku Flaska, utrudnia szybkie prototypowanie [45, 63].

Klasyczny stos LAMP (Linux, Apache, MySQL, PHP) przez lata stanowił fundament budowy stron i systemów CMS. Stabilność, prostota oraz fakt, że wszystkie komponenty są darmowe i open-source stanowią ogromny atut. Jednak w przypadku nowoczesnych aplikacji webowych LAMP ujawnia swoje ograniczenia. PHP działa synchronicznie, co utrudnia obsługę dużej liczby równoczesnych połączeń, a relacyjna baza MySQL gorzej radzi sobie z rosnącą ilością nieustrukturyzowanych danych. Stos ten również wymaga znajomości kilku różnych języków (PHP, SQL, JavaScript), co zwiększa barierę wejścia i spowalnia rozwój projektu [20, 64].

Stos MERN posiada szereg cech, które czynią go dobrym wyborem do budowy implementowanego systemu. Cały stos opiera się na jednym języku, co ułatwia naukę i upraszcza proces tworzenia aplikacji, a także obniża barierę wejścia dla nowych członków zespołu. Podejście komponentowe pozwala rozwijać aplikację o dodatkowe moduły oraz ułatwia tworzenie działających wersji MVP w krótkim czasie. Dodatkowo wszystkie elementy stosu są dostępne na otwartych licencjach. Eliminuje to koszty i pozwala swobodniej korzystać z technologii w środowisku akademickim [32, 37]. Stos ten ma też dużą społeczność i obszerną dokumentację, które wraz z ogromną liczbą materiałów edukacyjnych, takich jak tutoriale oraz kursy, ułatwiają szybkie rozwiązanie napotkanych problemów. Architektura Node.js i baza MongoDB ułatwiają rozwijanie aplikacji zarówno w pionie (dodawanie nowych funkcjonalności), jak i w poziomie (obsługa większej liczby użytkowników). Stos MERN wspiera również projekty, które zaczynają jako niewielkie systemy, a w przyszłości mogą być rozwijane do większych i złożonych aplikacji.



Rozdział 4

Architektura i model systemu IoTLab

System IotLab to aplikacja internetowa typu full stack. Strony internetowe zostały wykonane w technologii SPA (Single Page Application). Zwiększa to komfort pracy użytkownika i pozwala użytkownikowi odnieść wrażenie pracy w standardowym programie komputerowym, a nie w przeglądarce internetowej, ponieważ podczas przechodzenia pomiędzy różnymi modułami systemu następuje dynamiczne podmienianie fragmentów strony internetowej [21]. Skraca to czas odpowiedzi systemu oraz liczbę pełnych zapytań kierowanych do serwera jest zredukowana. Całość logiki interfejsu użytkownika obsługiwana jest po stronie przeglądarki. Komunikacja z serwerem odbywa się głównie za pomocą zapytań asynchronicznych. Wpływa to na bardziej płynne i responsywne działanie aplikacji.

Platforma dostarcza dwie główne strony internetowe. Stronę domową i kokpit użytkownika. Na stronie domowej, poza logowaniem do systemu, znajdują się istotne informacje oraz materiały graficzne. Kokpit użytkownika został natomiast podzielony na dwie sekcje. Umożliwia to równoczesne korzystanie z różnych funkcji systemu, pozwala szybko odnajdywać potrzebne informacje i wpływa pozytywnie na ergonomię pracy. Układ ten pozwala również dotrzeć użytkownikowi do najczęściej wykorzystywanych funkcji, jak rezerwacja sprzętu lub katalogu zasobów, w minimalnej ilości kroków.

Interfejs został tak zaprojektowany, aby automatycznie dostosowywał się do różnych rozdzielczości ekranów, a przez to był responsywny. Dzięki temu użytkownik może w równym stopniu korzystać z aplikacji na komputerze oraz urządzeniu mobilnym i mieć dostęp do tych samych funkcji oraz narzędzi w formie, która jest najbardziej czytelna i ergonomiczna dla tego typu urządzeń. Oddzielenie strony głównej od kokpitu użytkownika poprawia przejrzystość, bo na jednej stronie zapewniany jest dostęp do logowania oraz najważniejszych komunikatów, a kolejna strona jest przeznaczona do właściwej pracy. Ze względu na swój modułowy układ, możliwe jest łatwe dodawanie nowych funkcji i elementów, które rozbudują system.

4.1 Strona domowa

Pierwszy kontakt użytkownika z systemem odbywa się za pośrednictwem strony domowej. Ma ona kluczowe znaczenie dla odbioru całej aplikacji i pełni rolę wizytówki systemu, jednocześnie pełniąc rolę punktu wejścia do pełnego zakresu funkcjonalności. Strona domowa systemu IoTLab zaprojektowana jest w taki sposób, aby umożliwić użytkownikom szybki dostęp do najważniejszych funkcji i informacji jeszcze przed zalogowaniem do systemu. Poszczególne elementy są przedstawione w czytelny i przejrzysty sposób, tak aby minimalizować ryzyko dezorientacji na stronie oraz zwiększyć komfort i wygodę użytkowania.

Dzięki umieszczeniu na niej panelu logowania, użytkownik od razu rozpoznaje sposób, w jaki może uzyskać dostęp do zasobów. Brak konieczności poszukiwania odpowiedniej zakładki lub przechodzenia na kolejne strony usprawnia proces korzystania z systemu oraz eliminuje zbędne kroki wymagane do uzyskania pełnych funkcjonalności i zasobów.

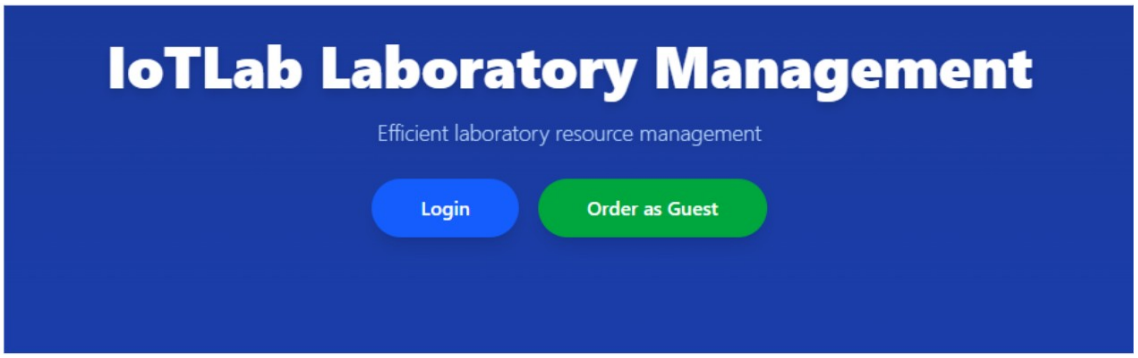
Element wizualny w postaci slidera uatrakcyjnia wygląd strony i nadaje jej nowoczesny charakter. Pełni też rolę reprezentatywną dla administratora systemu, bo pozwala w przystępny sposób zaprezentować sprzęt laboratoryjny, sale dydaktyczne lub ciekawe materiały. Takie rozwiązanie nie tylko wzmacnia estetykę, ale również buduje pozytywne pierwsze wrażenie, które ma duży wpływ na motywację do dlaczego korzystania z aplikacji.

Przekazywanie najważniejszych komunikatów jest możliwe dzięki panelowi ogłoszeń, który znajduje się pod sliderem. Użytkownik może w tym miejscu zapoznać się z informacjami o awariach, harmonogramie prac serwisowych oraz zmianach organizacyjnych, od razu po wejściu na stronę. Poprawia to komunikację miedzy administratorem systemu, a jego użytkownikami.

Zawartość slidera oraz panelu ogłoszeń jest edytowalna z poziomu panelu administratora. Wszystkie trzy elementy stanowią fundament do pozytywnego doświadczenia związanego z korzystaniem z IoTLab.

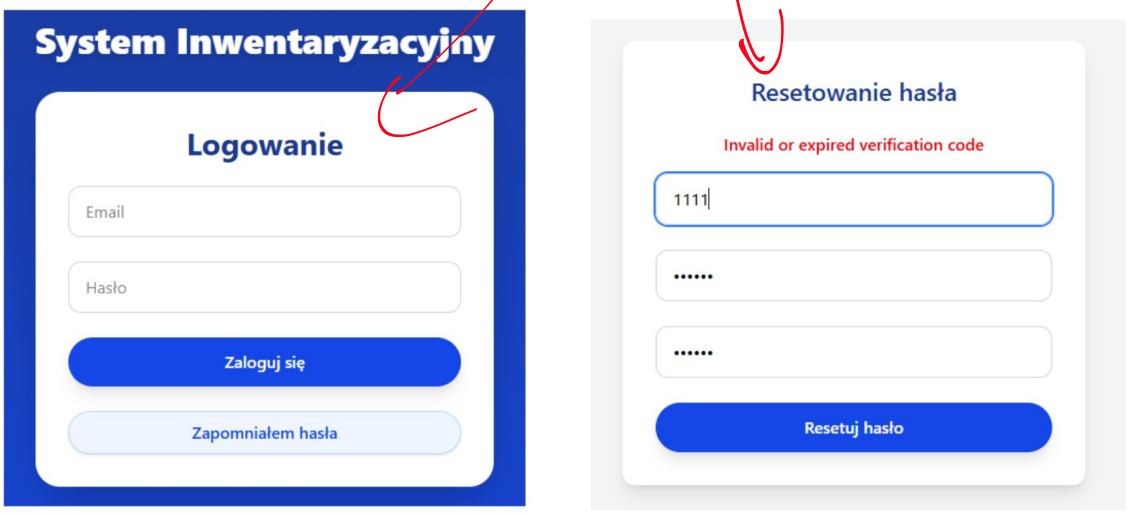
4.1.1 Panel logowania

Rysunek 4.1 przedstawia główny widok panelu logowania. Aby uzyskać dostęp do systemu, użytkownik musi się zalogować na swoje konto lub skorzystać z trybu gościa, który nie wymaga logowania.



Rysunek 4.1: Panel logowania [opracowanie własne]

Aby zalogować się do systemu należy podać swój adres e-mail oraz hasło. W celu zwiększenia bezpieczeństwa, wykorzystywane są mechanizmy hashowania haseł w bazie danych, a dane logowania weryfikowane są w oparciu o bezpieczne protokoły komunikacyjne (HTTPS). Jak widać na rys. 4.2, formularz logowania jest prosty i minimalistyczny. Ogranicza się do dwóch pól, przycisku logowania i oraz kolejnego, do resetu hasła. W zależności od przypisanej do konta roli, użytkownik otrzymuje dostęp do odpowiedniego kokpitu, który zawiera dostęp do najpotrzebniejszych dla niego funkcji.



Rysunek 4.2: Panel logowania oraz resetowania hasła [opracowanie własne]

Użytkownik, który nie posiada konta lub nie może się zalogować do systemu, nadal może przeglądać dostępne zasoby oraz składać zamówienia bez konieczności rejestracji. W tym celu należy wybrać opcję przedstawioną na rys. 4.1 jako „Order as Guest", po czym użytkownik zostaje przeniesiony do menu kategorii. Rozwiązanie to dodatkowo zwiększa dostępność systemu i umożliwia szybkie korzystanie z jego podstawowych funkcji. W przypadku utraty hasła przewidziano dwie możliwości jego odzyskania:

- kontakt z administratorem systemu – administrator ma możliwość ręcznego ustawienia nowego hasła dla wybranego konta;
- procedura resetu hasła z użyciem kodu uwierzytelniającego – na adres e-mail przypisany do konta wysyłany jest jednorazowy kod. Po jego poprawnym wprowadzeniu oraz podaniu nowego hasła system aktualizuje dane konta, umożliwiając użytkownikowi korzystanie z nowego hasła podczas logowania. Okno zmiany hasła widoczne jest na rys. 4.2.

4.1.2 Slider

Na rys. 4.3 przedstawiony jest slider. Urozmaica on interfejs i przełamuje stateczne treści, przyciągając jednocześnie uwagę.

Jest to element strony głównej pozwalający na promowanie laboratorium oraz na udostępnienie informacji związanych z jego działalnością. Administrator posiada możliwość umieszczenia materiałów promocyjnych w postaci zdjęć oraz ustalenia kolejności ich wyświetlania w sliderze. Zdjęcia posiadają możliwość przypisania linków przenoszące użytkownika do innych stron np. kanału YouTube laboratorium lub na stronę wydziału z informacjami o organizowanym konkursie. Pozwala to na promowanie laboratorium poprzez informowanie osób odwiedzających stronę domową systemu o jego działalności, wydarzeniach lub organizowanych konkursach.

Obrazy przewijają się automatycznie z możliwością ręcznej nawigacji (poprzez strzałki z prawej i lewej strony slidera), a zdjęcia dostosują się do rozdzielczości ekranu.

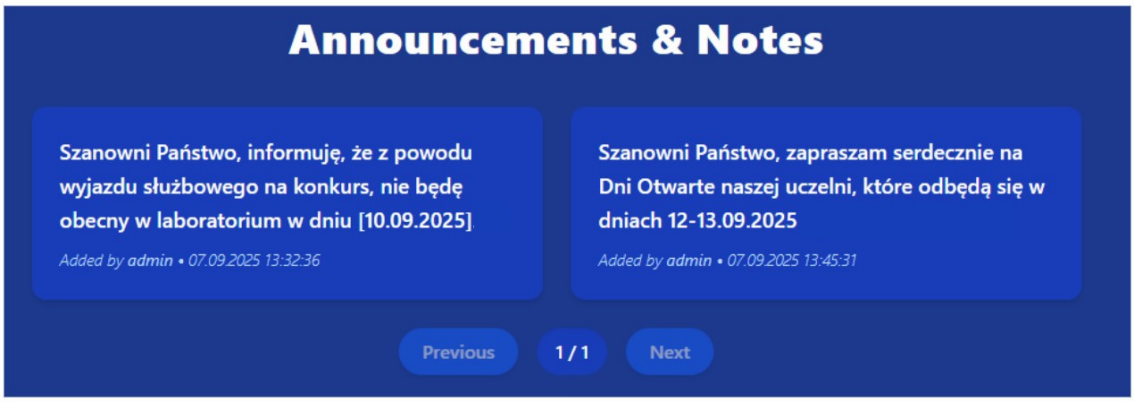


Rysunek 4.3: Slider umieszczony nad modułem ogłoszeń [opracowanie własne]

4.1.3 Notatki

Jest to element strony głównej, którego zadaniem jest informowanie użytkowników o ważnych wydarzeniach mających wpływ na pracę laboratorium np. odwołanie zajęć, awarie, zmiana godzin pracy. Eliminuje to problem rozproszonej komunikacji, gdy administratorowi systemu zależy na dotarciu do dużej liczby osób. Jak widać na rys. 4.4, narzędzie to dostarcza wirtualną tablice ogłoszeń dzięki czemu przekazywanie informacji jest szybkie i nie wymaga od użytkowników fizycznej obecności w sali laboratoryjnej.

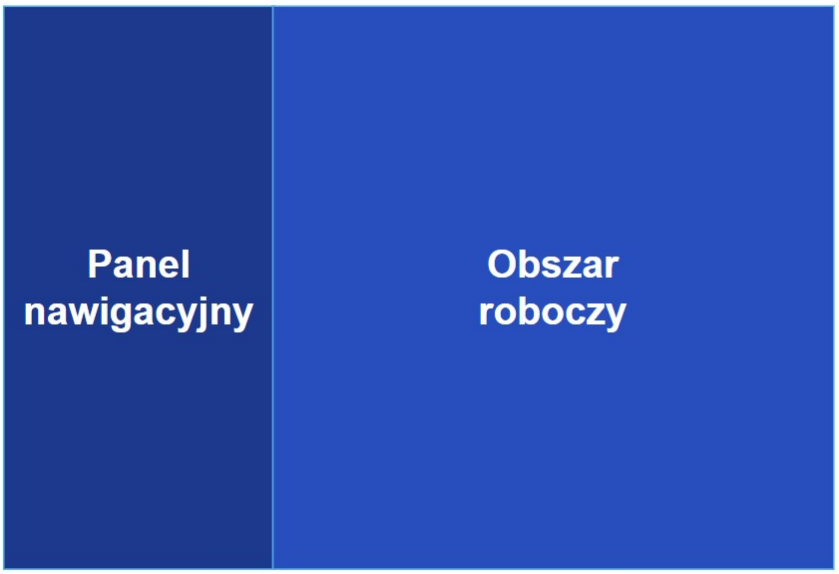
Notatki są wyświetlane w kolejności dodania, co pozwala śledzić bieżące zmiany. Najwcześniejsza notatka jest umieszcza na samej górze widoku notatek. Moduł notatek pozwala wyróżniać notatki zwierające ważniejsze informacje. Notatki są przygotowywane w dedykowanym module w panelu użytkownika.



Rysunek 4.4: Moduł ogłoszeń [opracowanie własne]

4.2 Kokpit użytkownika

Głównym środowiskiem pracy w systemie IoTLab jest kokpit użytkownika. Zawiera on niezbędne widoki do zarządzania zasobami, zamówieniami oraz komunikacją, przy jednoczesnej minimalizacji kroków do wykonania typowych zadań. Interfejs został zaprojektowany w taki sposób, aby był spójny i zgodny z zasadami hierarchii informacji. Jak widać na rys. 4.5, po lewej stronie widoczny jest stały panel nawigacyjny. Zapewnia on szybki dostęp do najważniejszych modułów, takich jak: Categories, Orders, Users, Profile, Notes, Slider. Przestrzeń robocza, która znajduje się po prawej stronie, wykorzystuje podejście SPA. Zwiększa płynność pracy i wyeliminować pełne przeładowanie strony, przy przejściu do kolejnego modułu. Prezentowane są w niej treści i formularze danego modułu.



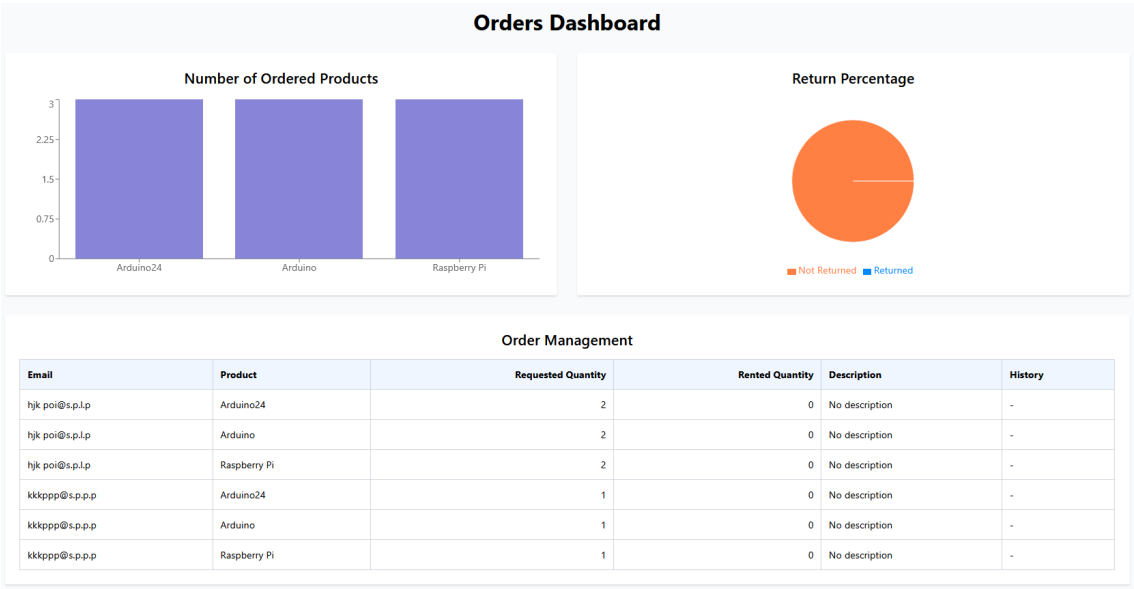
Rysunek 4.5: Schemat układu elementów w kokpitu użytkownika [opracowanie własne]

Dostęp do kokpitu jest uprawnieniowy, a widok kontekstowy. Dany użytkownik widzi tylko te moduły do których ma dostęp zgodnie z przypisaną przez administratora rolę. Operacje, które ingerują w dane, są potwierdzane komunikatami zwrotnymi oraz walidowane po stronie interfejsu, jak i serwera. Ogranicza to ryzyko błędów i zwiększa świadomość użytkownika. Aby dodatkowo zwiększyć ergonomię pracy, przy listach i tabelach zastosowano możliwość filtrowania oraz sortowania, formularze mają przewidywalny układ, elementy priorytetowe, taki jak statusy zamówień lub ważne notatki, są odpowiednio wyróżniane, a w przypadku konfliktów terminów wypożyczeń sprzętu, pojawiają się informatory.

Tak jak strona domowa, kokpit użytkownika jest responsywny i skalowany. Układ elementów zmienia się i dostosowuje do rozmiaru urządzenia. Między modułami występują wspólne wzorce nawigacyjne oraz podobne komponenty interfejsu, tak aby ułatwić wdrożenie nowych użytkowników i skrócić czas ich szkolenia.

4.2.1 Moduł Statistics

Jednym z modułów, znajdujących się w panelu nawigacyjnym jest moduł Statistic, przedstawiony na rys. 4.6. Wykresy oraz tabele mają charakter interaktywny, co ułatwia analizę dużych zbiorów danych. Kolorystyka jest tak dobrana, aby ułatwić szybkie odczytanie kluczowych informacji.



Rysunek 4.6: Moduł Statistics [opracowanie własne]

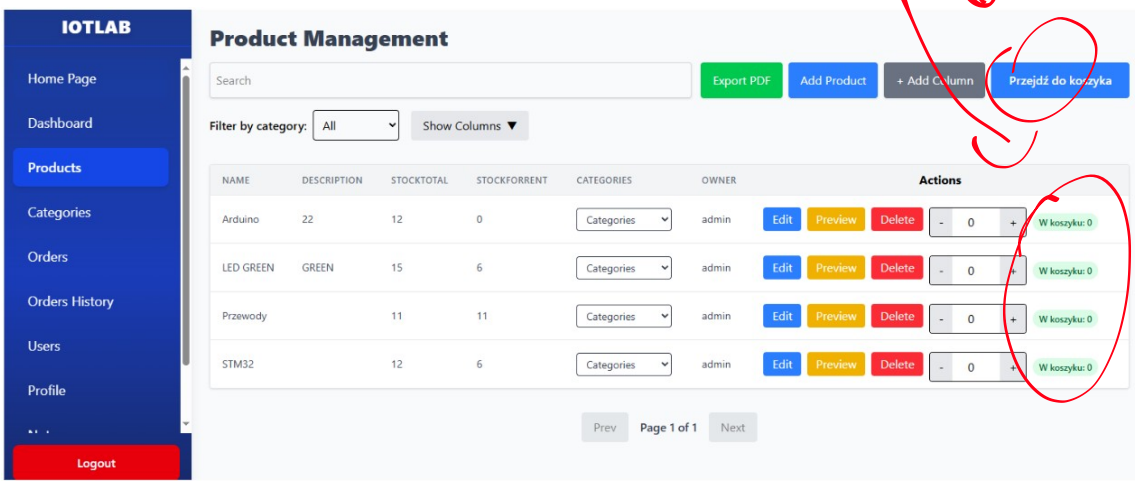
Moduł ten pozwala przeglądać statystki pracy systemu. Użytkownik ma dzięki niemu wgląd w następujące informacje:

- liczba obiektów w systemie;
- liczba wypożyczonych obiektów;
- najczęściej wypożyczane obiekty;
- najpopularniejsza kategoria;
- liczba przedmiotów użytkownika;
- liczba zamówień;
- liczba dorzuconych zamówień;
- średni czas trwania wypożyczenia.

Dzięki tym informacją użytkownik może podejmować optymalne decyzje związane z zarządzaniem laboratorium dydaktycznym np. poprzez zwiększenie ilości przedmiotów najchętniej wypożyczanych przez studentów. Statystki prezentują informacje w postaci wykresów i tabel. Użytkownik może przełączać się pomiędzy rożnymi informacjami. Pozwala to na intuicyjna i szybkie wydobywanie informacji [67].

4.2.2 Moduł Objects

Kolejny moduł, który jest dostępny z panelu nawigacyjnego to moduł Objects i jest on widoczny na rys. 4.7. Pozwala on na dodawanie, przeglądanie, modyfikowanie i usuwanie obiektów do systemu. Z tego względu jest jednym z centralnych modułów całego systemu, bo bezpośrednio odpowiada za odwzorowanie zasobów laboratorium w środowisku cyfrowym. Jego głównym zadaniem jest utrzymanie porządku i aktualności bazy danych. Zastosowanie tabel i widoków kart zwiększa czytelność. System pozwala na ukrywanie wrażliwych zasobów czy też kontrolowanie widocznych i dostępnych elementów. Użytkownik może sortować elementy po każdej wprowadzonej kolumnie. Sortownie odbywa się w kolejności leksykograficznej.



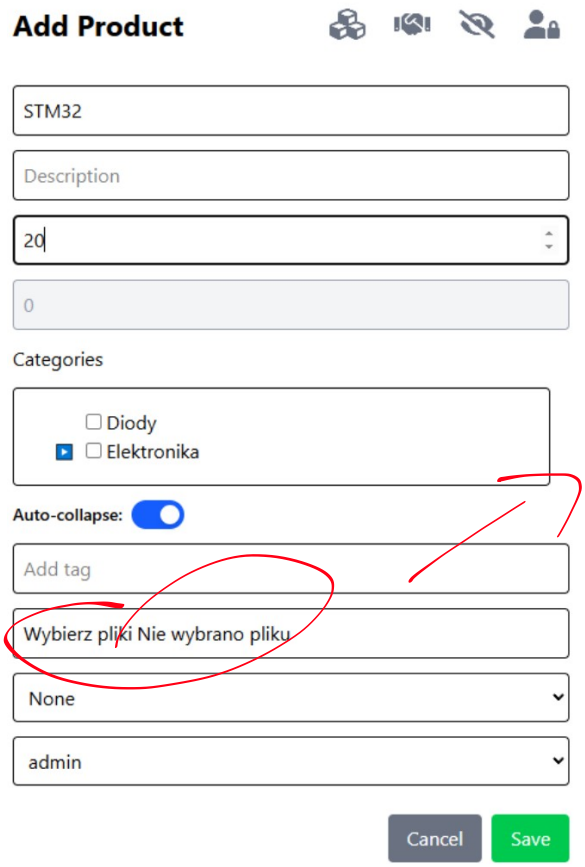
Rysunek 4.7: Moduł Objects [opracowanie własne]

Dodawanie obiektu

W celu dodania nowego obiektu do systemu użytkownik musi wybrać opcję „Add Object", widoczną na rys. 4.7. Następnie w oknie, przedstawionym na rys. 4.8, musi wprowadzić dane wymagane przez system do poprawnego operowania na obiekcie.

Do pól obligatoryjnych należy: nazwa obiektu oraz liczba obiektu.

Do pól opcjonalnych zalicza się: kategoria, liczba przeznaczona do wypożyczenia, opis, tag, pliki i kolumny dodatkowe.



Rysunek 4.8: Okno dodawania obiektu [opracowanie własne]

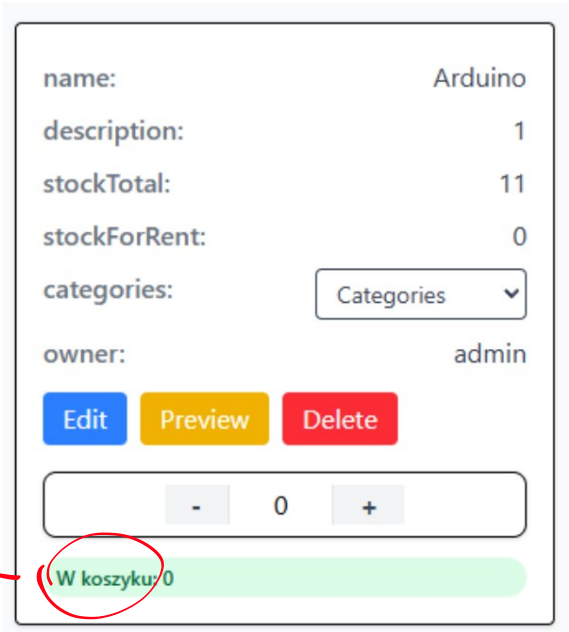
U góry okna przedstawionego na rys. 4.8, widoczne są ikony parametry obiektu. Każdy obiekt w systemie może posiadać cztery cechy. Patrząc się na rys. 4.9 zaczynając od lewej, są to:

- tagowalność – pozwala na nadawanie indywidualnych tagów każdemu wystąpieniu danego obiektu w systemie. Cecha ta pozwala na nadawanie unikalnych numerów obiektów takim jak meble czy mikrokontrolery. Pozwala to na rozróżnienie elementów tego samego obiektu. Cechy tej nie należy nadawać elementom nierozróżnialnym takim jak diody;
- wypożyczalność – pozwala na dodawanie obiektu do koszyka przez studenta. Obiekt niewypożyczany nie może zostać dodany do koszyka, ale może być widoczny. Pozwala to na udostępnianie np. plików PDF zawierających instrukcje lub tutoriale;
- widzialność – pozwala na wyświetlanie obiektu w widoku obiektów studenta. Obiekt pozbawiony tej cechy nie jest widoczny dla studentów i nie może zostać dodany do koszyka;
- prywatność - cecha obiektu, która pozwala ukryć jego widoczność przed innymi wykładowcami. Właściciel może dodać zasoby, do których dostęp będzie posiadał wyłącznie on oraz administrator systemu. Student nie może wyświetlić obiektu posiadającego tę cechę.



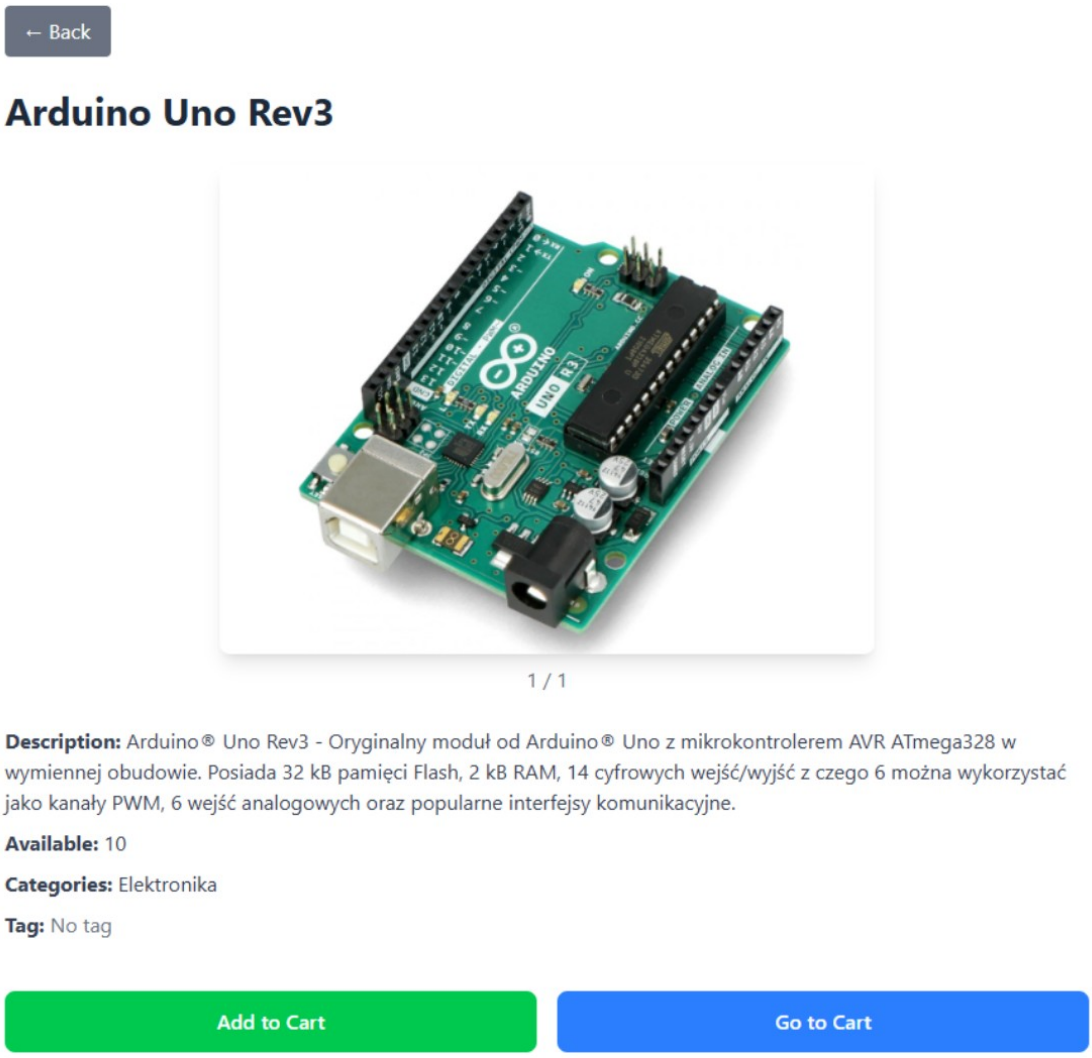
Rysunek 4.9: Ikony parametrów obiektu [opracowanie własne]

W przypadku urządzeń mobilnych, produkty nie są wyświetlane w tabeli, lecz jako karty, co jest przedstawione na rys. 4.10.



Rysunek 4.10: Widok kart obiektu na urządzeniu mobilnym [opracowanie własne]

Każdy obiekt posiada widok szczegółowy, przedstawiony na rys. 4.11. Pozwala on na łatwe przeglądanie jego właściwości oraz materiałów z nim związanych. Widok ten posiada wbudowany slider dzięki czemu przeglądnie zdjęć obiektu jest łatwiejsze i szybsze niż pobieranie.



Rysunek 4.11: Szczegółowy widok produktu [opracowanie własne]

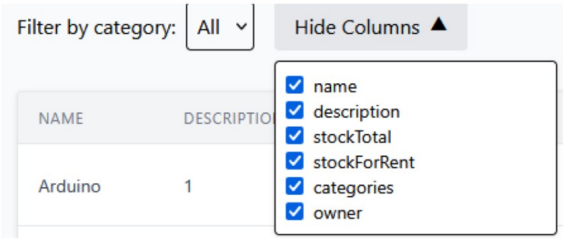
Dodawanie plików

(tutaj będzie jeszcze tekst)

Menadżer kolumn

W celu umożliwienia użytkownikom symulowania dowolnego obiektu laboratorium system posiada menadżer kolumn bazy danych, który jest przedstawiony na rys. 4.12. Narzędzie to pozwala na dodawanie specjalnych cech obiektom. Cechy te posiadają wybrane elementy i nie są obowiązkowe.

Kolumny opisujące cechy dodatkowe są przechowywane jako mapa posiadająca klucz i wartość. Dzięki takiemu rozwiązaniu użytkownik może dodać dowolne informacje o obiekcie. Pozwala to uzyskiwać dokładne odwzorowanie różnorodnych elementów laboratorium w systemie. Dodatkowo narzędzie to pozwala zapisywać informacje pomocnicze np. date oddana sprzętu do naprawy albo kiedy sprzęt musi zostać poddany utylizacji. Menadżer pozwala na dodawanie, usuwanie, edycje oraz ukrywanie kolumn. Ukryte kolumny stają się niewidoczne w panelu przeglądu obiektów oraz nie są uwzględniane podczas generowania dokumentów wspomagających inwentaryzacje. Pozwala to na dostosowywanie widoków tabeli obiektów do potrzeb użytkowników jak również na generowanie dokumentów inwentaryzacyjnych zawierających różne wybrane przez użytkownika informacje.



Rysunek 4.12: Widok managera kolumn [opracowanie własne]

4.2.3 Moduł Categories

Na rys. 4.14 przedstawiony jest moduł Categories. Umożliwia on zarządzanie kategoriami oraz strukturą drzewa kategorii. Głównym celem kategorii jest grupowanie produktów w celu łatwiejszego wyszukiwania i zarządzania systemem. Kategorie pozwalają na budowanie dodatkowych funkcjonalności systemu. Przykładem takiego zastosowania może być utworzenie kategorii „Tutoriale", w której przechowywane są obiekty niewypożyczalne zawierające pliki PDF z materiałami dydaktycznymi dla studentów.

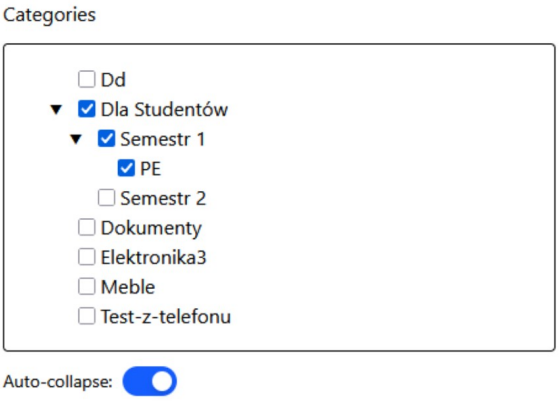


Rysunek 4.13: Widok modułu Category [opracowanie własne]

Drzewo kategorii

Drzewo kategorii, widoczne na rys. 4.14, to narzędzie umożliwiające budowę hierarchicznej struktury przechowującą kategorie i podkategorie. Uprawnienia do tworzenia kategorii posiada administrator systemu. Może on dodawać nowe kategorie oraz ich podkategorie, które razem budują drzewo kategorii obiektów. Pozwala to studentom na łatwą nawigacje i uzyskiwanie dostępu do tych elementów.

Kategoria może być niewidoczna wówczas należące do niej obiekty nie są dostępne dla studentów. Administrator może zbudować drzewo, które posiada gałęzie przeznaczone tylko dla wykładowców np. przechwytujące wyniki badań lub specjalne materiały.



Rysunek 4.14: Drzewo kategorii podczas dodawania obiektu [opracowanie własne]

Moduł zarządzania kategoriami umożliwia administratorowi pełną kontrolę nad strukturą systemu. W głównym widoku prezentowana jest lista kategorii w formie drzewa. Użytkownik może w prosty sposób dodawać nowe kategorie, edytować istniejące lub usuwać niepotrzebne. Dodatkowo każdej kategorii można nadać opis oraz zdecydować, czy będzie widoczna dla studentów. Przejrzysty podział widoku oraz formularz do edycji ułatwiają zarządzanie nawet rozbudowaną strukturą. System automatycznie odświeża listę po każdej operacji, co zapewnia płynność pracy.

4.2.4 Moduł Orders

Jest to moduł systemu pozwalający użytkownikowi na przeglądanie zamówień złożonych przez studentów. Administrator może przeglądać wszystkie zamówienia natomiast wykładowca jedynie te dotyczące przedmiotów, których jest właścicielem.

Moduł wyświetla poszczególne zamówienia jako karty. W zamówieniu znajdują się informacje o studencie, który złożył prośbę o wypożyczenie oraz lista sprzętu, który ma zostać wypożyczony wraz z liczbą i datami wypożyczenia i zwrotu. Użytkownik może edytować ilość sprzętu oraz daty wypożyczenia w zamówienia. Każdy element zamówienia jest rozpatrywany osobno dzięki czemu właściciel sprzętu może precyzyjnie określić warunki wypożyczenia.

Wykładowca może nie zgodzić się na wypożyczenie poprzez odrzucenie zamówienia. Spowoduje to wysłanie do studenta widmowości e-mail z informacją o odrzuceniu zamówienia. W przypadku zmodyfikowania zamówienia przez właściciela sprzętu system wysyła wiadomość e-mail do studenta z informacją o wprowadzonych zmianach w zamówienia oraz z prośbą o wysłanie maila do wykładowcy z informacją czy zgadza się na nowe warunki lub czy rezygnuje z zamówienia.

Zamówienie zaakceptowane przez wykładowce i studenta zostaje zatwierdzone w systemie. Zamówienie zaakceptowane czeka na odbiór sprzętu przez studenta. System pozwala wówczas wygenerować dokument zawierający umowę wypożyczania. Zamówienie po wypożyczeniu sprzętu oczekuje na jego zwrot. Wypożyczanie sprzętu powoduje zmniejszenie liczby dostępnego sprzętu w tabeli obiektów. Po zwrocie sprzętu przez studenta wykładowca oznacza wypożyczanie jako zakończone. Powoduje to powrót sprzętu do listy obiektów i daje możliwość wypożyczenia go przez kolejnych studentów.

Użytkownik podczas rozpatrywania zamówienia otrzymuje informacje od systemu o tym czy inni studenci nie złożyli zamówień na dany sprzęt. Pozwala to na podejmowanie sprawiedliwszych

decyzji i eliminowanie sytuacji w których jeden student dostanie cały sprzęt, a zabranie go dla pozostałych ze względu na nieświadomości o innych zamówieniach podczas zatwierdzania zamówienia.

Zmiana ilości sprzętu w zamówieniu powoduje wysłanie do studenta wiadomości e-mail z informacją o dokonanych zmianach i prośbą o wysłanie wiadomości e-mail do właściciela sprzętu z odpowiedzią czy akceptuje wprowadzone zmiany i czy nadal jest zainteresowany zamówieniem. Przewidziane statusy zamówień to:

- Pending – zamówienie oczkujące na rozpatrzenie;
- Changed – zmodyfikowane zamówienie;
- Approved – zatwierdzone zamówienie;
- Prepared – zamówienie gotowe do odbioru;
- Rented – zamówienie wypożyczone;
- Returned – zamówienie zwrócone;
- Rejected – odrzucone zamówienie.

Wypożyczanie

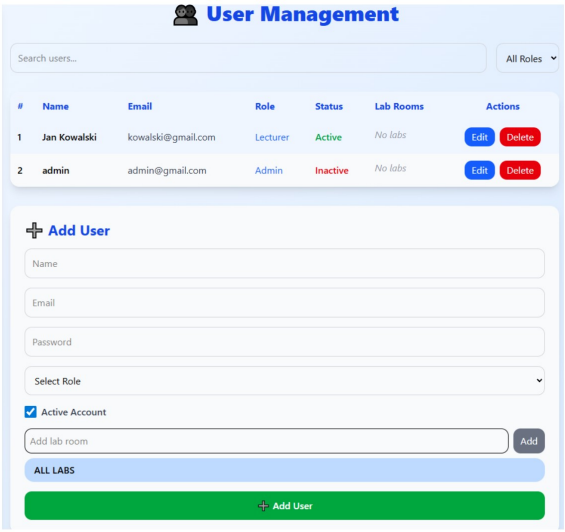
Wypożyczanie nie wymaga konta użytkownika co znacząco ułatwia cały proces. Klient może przeglądać produkty dostępne do wypożyczenia w przeznaczonym do tego oknie systemu. Tabela obiektów pozwala sortować je po kategorii i nazwie. Wybranie danego obiektu wyświetla jego widok szczegółowy, gdzie użytkownik może sprawdzać jego opis materiał dodatkowe.

Użytkownik posiada koszyk podobny do tych w sklepach internetowych. Może umieszczać w nim wybrane obiekty. Musi on określić ilość oraz przedział czasowy, w którym chce wypożyczyć sprzęt laboratoryjny. Następnie podaje swoje dane: imię, nazwisko oraz numer indeksu. Są one wymagane w celu identyfikacji studenta oraz do wygenerowania dokumentów umowy wypożyczenia sprzętu. Student musi podać również adres e-mail, który pozwoli na komunikację z użytkownikiem systemu oraz na otrzymywanie informacji o statusie zamówienia. Ostatnim krokiem jest przepisanie wygenerowanego kodu w celu potwierdzenia złożenia zamówienia

4.2.5 Moduł Users

Na rys. 4.15 przedstawiony jest to moduł Users. Pozwala on administratorowi na zarządzanie użytkownikami. Użytkownik nie może sam utworzyć konta. Każde konto musi zostać utworzone przez administratora. Może on tworzyć konta wykładowców, jak i innych administratorów poprzez wybór roli konta w systemie. Konto w module Users może zostać zmodyfikowane. Administrator może zmienić nazwę konta, jego adres e-mail hasło oraz przypisane sale. Konto może zostać również usunięte albo zawieszone. Zawieszone konto istnieje w systemie, ale jego właściciel nie może zalogować się nim do systemu.

System posada wbudowane konto admin, które jest dodane podczas pierwszego uruchomienia systemu. Po zalogowaniu do konta admin można zmienić jego email i hasło. Możliwa i zalecana jest również opcja utworzenia nowego konta administratora i wyłączenie domyślnego. Nie można wyłączyć wszystkich kont administratorów, przynajmniej jedno konto musi być zawsze aktywne.



Rysunek 4.15: Widok główny panelu Users pozwala na sortowanie użytkowników i łatwe przeglądanie parametrów ich kont [opracowanie własne]

Rodzaje użytkowników

Platforma IoTLab wyróżnia trzy rodzaje użytkowników: administrator, wykładowca oraz student. Jedynie administrator i wykładowca posiadają własne konta, użytkownik typu student nie posiada dedykowanego konta w systemie.

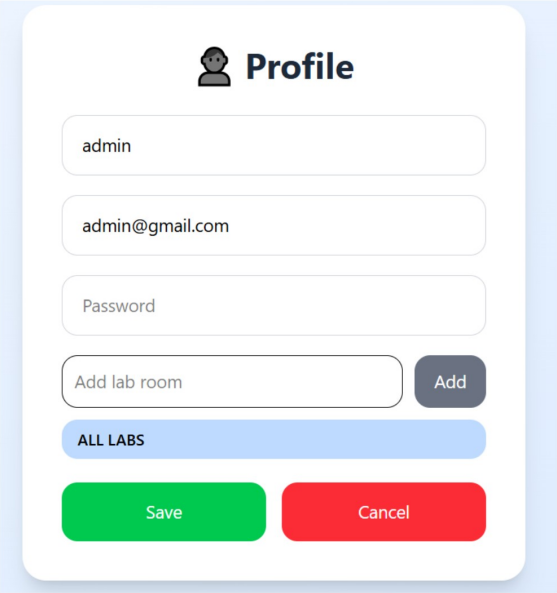
Administrator to użytkownik posiadający konto o najwyższych uprawnieniach i możliwościach. Kompetencje administratora pozwalają na dostęp do specjalnych modułów związanych z tworzeniem innych kont, dodawaniem drzew kategorii oraz promocją sali laboratoryjnej. Administrator może również edytować dane kont i zmieniać właścicieli przedmiotów.

Wykładowca to konto zwykłego użytkownika systemu. Pozwala ono na dostęp do modułów związanych z obiektami, statystykami, wypożyczeniami i ich historią oraz notatkami. Wykładowca ma przypisaną sale laboratoryjną, za którą odpowiada. Obiekty, które umieści w systemie są przypisane do jego konta, a inni wykładowcy nie mogą ich edytować. W przypadku zapomnienia hasła wykładowca może zmienić je sam korzystając z kodu OTP lub poprosić o jego zmianę administratora.

Ostatnim rodzaje użytkownika systemu IoTLab jest student. Nie ma on dostępu do żadnego modułu oferowanego przez system. Student może jedynie przeglądać obiekty udostępnione przez wykładowców w drzewie kategorii. Wybrane produkty może umieścić w koszyku i złożyć zamówienie.

4.2.6 Moduł Profile

Jest to moduł pozwalający użytkownikowi zmieniać parametry swojego konta. Możne on przy pomocy tego modułu zmienić hasło nazwę i adres e-mail. Jego wygląd przedstawia rys. 4.16. Użytkownik może również modyfikować listę sal, których jest opiekunem.



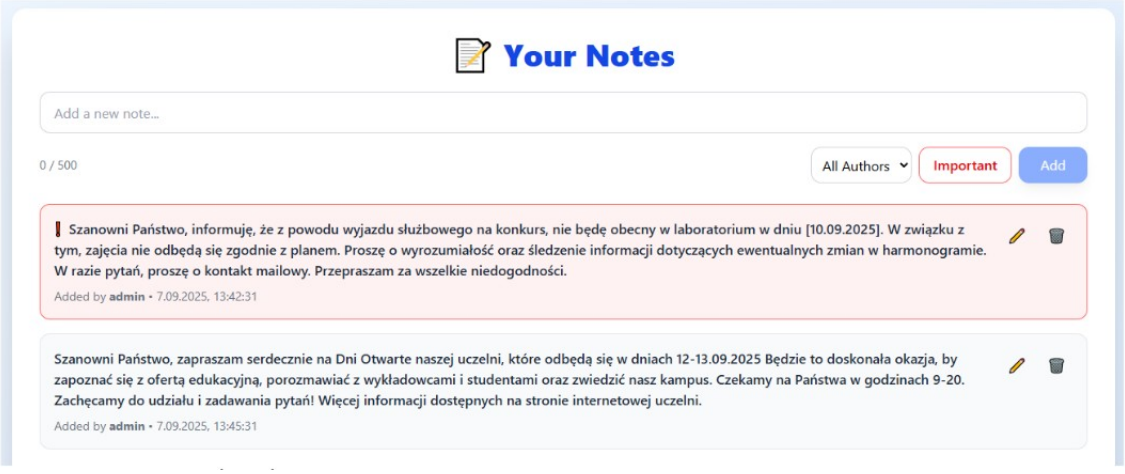
Rysunek 4.16: Widok modułu Profile [opracowanie własne]

4.2.7 Moduł Notes

Moduł notatek, przedstawiony na rys. 4.17, pozwala na zarządzanie notatkami wyświetlanymi na stronie domowej systemu. Panel roboczy pozwala na generowanie, edytowanie oraz usuwanie notatek.

Notatka może zostać wyróżniona. Użytkownik dokonuje tego nadając notatce flagę *important*. Notatka posiadające takie wyróżnienie jest zawsze wyświetlana na górze razem z innymi notatkami *important*, natomiast zwykłe notatki, również te dodane później niż notatka z flagą *important*, będzie wyświetlana niżej. Status *important* może zostać odebrany notatce, która go posiada oraz można nadać go standardowej notatce. Treść notatki może zostać zmodyfikowana, a nieaktualna notatka może zostać usunięta.

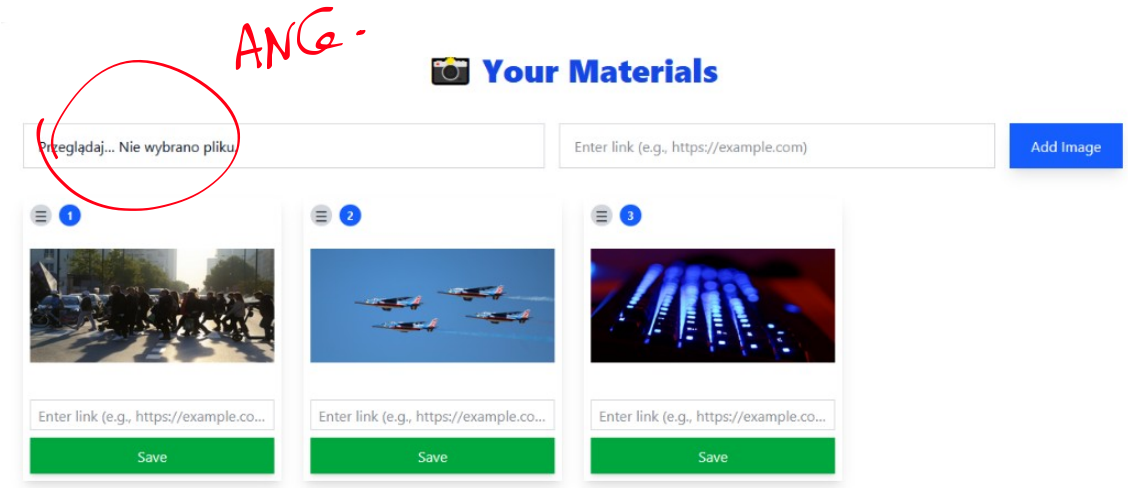
Moduł pozwala również wyświetlać notatki konkretnego użytkownika dzięki czemu odnajdowane konkretnych notatek jest łatwiejsze. Treść notatki może być edytowana w dowolnym czasie jednak jej długość nie może przekroczyć 500 znaków. Ponadto każda notatka jest automatycznie podpisywana nazwą użytkownika, który ją umieścił oraz posiada datę dodania jej do systemu



Rysunek 4.17: Widok modułu Notes [opracowanie własne]

4.2.8 Moduł Slider

Jest to moduł pozwalający na zarządzanie Sliderem znajdującym się na stronie głównej systemu. Jak widać na rys. 4.18, umożliwia on dodawanie zdjęć i dołączania do nich opcjonalnych linków. Użytkownik może ustawiać kolejność slajdów przeciągając je i upuszczając w dowolnym miejscu w kolejce slajdów. Link dodany do zdjęcia może zostać zmieniony. Slajdy nieaktualne można usunąć z kolejki.



Rysunek 4.18: Widok modułu Slider [opracowanie własne]

4.2.9 Strony poboczne

Menu Kategorii

Jak widać na rys. 4.19, menu kategorii to drzewo nawigacyjne pozwalające studentom łatwo przechodzić do obiektów przydzielonych danej kategorii. Student może przejść do wybranej kategorii lub jej podkategorii dzięki rozwijanej liście. Menu jest budowane przez administratorów w dedykowanym module systemu. W menu kategorii wyświetlane są jedynie kategorie z wybraną opcją widoczności. Student nie może przeglądać kategorii, które nie są dla niego udostępnione.

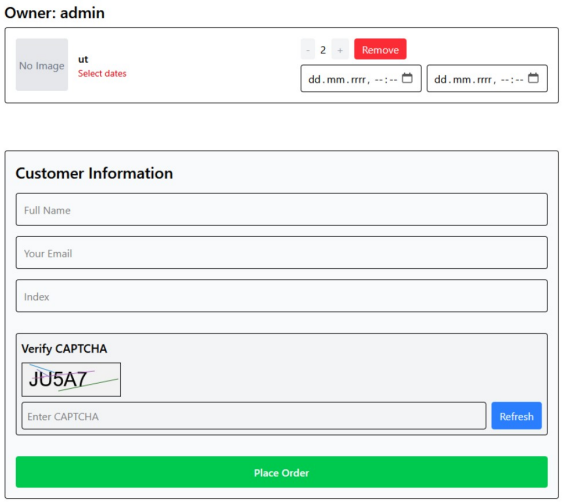


Rysunek 4.19: Menu kategorii z produktami widocznymi dla studenta [opracowanie własne]

Koszyk

Jest to strona, która wyświetla sprzęt wybrany przez studenta w celu jego wypożyczenia. Jak widać na rys. 4.20, składa się ona z listy przedmiotów, w której student posiada możliwość dostosowywania liczby danego przedmiotu oraz przedział czasowy, w którym chce dokonać wypożyczenia Drugą jej cześć strony stanowi formularz z danymi niezbędnymi do złożenia zamówienia:

- imię i nazwisko;
- adres email;
- numer indeksu.

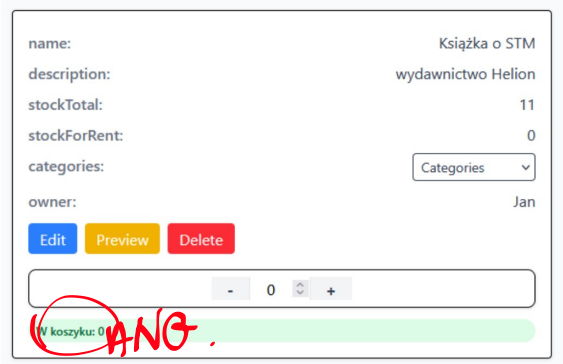


Rysunek 4.20: Widok koszyka studenta podczas składania zamówienia [opracowanie własne]

Student w celu złożenia zamówienia musi uzupełnić poprawnymi danymi wszystkie pola formularza. Wymagane jest również przepisanie kodu CAPTCHA. Mechanizm ten zapobiega próbie złośliwego wysyłania dużych liczby zamówień przez jednego studenta.

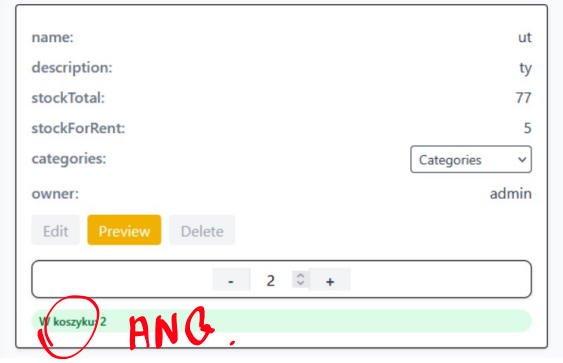
Zmiana właściciela i edycja obiektu

Wykładowca, który wprowadza obiekty do systemu staje się ich właścicielem i może ona dowolnie edytować ten obiekt. Ma on dostęp pełnej jego edycji, tak jak przedstawiono na rys. 4.21.



Rysunek 4.21: Widok edycji obiektu przez administratora [opracowanie własne]

Oznacza to, że próba wypożyczenia obiektu utworzy zamówienie skierowane do tego wykładowcy. System posiada możliwość zmiany właściciela w celu przekazania odpowiedzialności za obiekty systemu w sytuacji, kiedy wykładowca np. przestaje być opiekunem laboratorium. Dokonanie zmiany właściciela jest możliwe wyłącznie dla obiektów niewypożyczonych. W przypadku, kiedy obiekt został wypożyczony należy zaczekać na jego zwrócenie, ponieważ właściciela i studenta obowiązuje umowa. Jeżeli obiekt utracił cechę widoczności, a zostały złożone dotyczące go zamówienia student jest informowany wiadomością e-mail o takiej zmianie i jest poproszony o próbę wypożyczenia obiektu od innych wykładowców. Wykładowca może edytować wyłącznie swoje obiekty i przy próbie edycji obiekty zobaczy ekran jak na rys. 4.22.



Rysunek 4.22: Widok braku możliwości edycji obiektu, którego nie jest się właścicielem [opracowanie własne]

Komunikacja e-mail

System do komunikacji ze studentami i wykładowcami wykorzystuje automatyczne wiadomości e-mail. System IoTLab w swojej konfiguracji posiada własne konto e-mail, przy użyciu, którego wysyłane są wiadomości do użytkowników.

System nie pełni roli komunikatora. Wysyła automatyczne powiadomienia w oparciu o przygotowane wcześniej schematy wiadomości. System uzyskuje informacje kontaktowe dzięki obowiązkowemu podaniu adresu e-mail przy tworzeniu konta oraz podczas składania zamówienia. Pozwala to systemowi na logiczne połączenie studenta i właściciela obiektu. Prośba o wypożyczenie zostanie wysłana na adres e-mail wykładowcy oraz odnotowana w jego panelu zarządzania zamówieniami. Student również jest informowany o procesie swojego zamówienia za pośrednictwem poczty e-mail. Wykładowca może komunikować się ze studentem bezpośrednio poprzez swoją pocztę e-mail ale częste i powtarzalne wiadomości takie jak: zgoda, odmowa, zażądanie zwrotu może wysłać z poziomu systemu co znacząco ułatwia zarządzanie wypożyczeniami.

4.3 Proces wypożyczenia sprzętu

Student, który chce wypożyczyć sprzęt musi wybrać opcję, "Order as a guest" w panelu logowania. Zostanie wówczas przeniesiony do widoku menu kategorii, skąd może wybrać kategorii sprzętu, który go interesuje. Następny krok to odszukanie interesujących go przedmiotów z tabeli lub widoku kart, w przypadku pracy na urządzeniu mobilnym. Student po znalezieniu sprzętu musi dodać go do swojego koszyka.

Znalezione sprzętu jest łatwe dzięki możliwości sortowania po dowolnej kolumnie oraz kategorii. Dodatkowo widok posiada wyszukiwarkę, która pozwala znaleźć szukany przedmiot po jego nazwie.

Po dodaniu sprzętu do koszyka może wrócić do widoku tabeli przedmiotów lub do menu kategorii. Gdy student doda do koszyka wszystkie przedmioty, które chce wypożyczyć, wypełnia

formularz wypożyczenia. Konieczne jest podanie daty planowanego wypożyczenia i deklarowanego zwrotu sprzętu. Dodatkowe wymagane jest wypełnienie danych osobowych i kontaktowych. Ostatnim korkiem jest rozwiązanie RECAPCH chroniącej przed botami internetowymi i złośliwymi studentami.

W przypadku poprawnego wypełnienia wszystkich kroków system umieszcza zamówienie w module Orders. Zamówienie jest widoczne dla właściciela sprzętu. Wykładowca podejmuje decyzję czy zgadze się na wypożyczenie. Może zgodzić się na wypożyczenie całości lub jej odrzucenie. Posiada również możliwość edytowania parametrów zamówienia, takich jak data wypocznie, zwrotu lub ilość sprzętu. Każdy przedmiot może być rozpatrywany osobno.

Zmiana parametrów zamówienia, skutkuje automatycznym wysłaniem przez system wiadomości e-mail do studenta z informującą o zmianach i z prośbą o zaakceptowanie tych zmian lub zrezygnowanie z zamówienia. Jeśli student zgodzi się na zmiany lub jego zamówienie zostanie od razu zaakceptowane, może wtedy, zgłosić się do wykładowcy po odbiór sprzętu.

Podczas odbioru sprzętu wykładowca drukuje umowę wypożyczenia wygenerowaną przez system. Na umowie znajdują się wyszczególnione przedmioty, które znajdowały się w zamówieniu, dane studenta oraz czas obowiązywania umowy.

```
1 (tutaj będzie listing z umową)
```

Listing 1: Umowa

Po podpisaniu umowy i wydaniu sprzętu wykładowca zaznacza w systemie informacje o zatwierdzeniu wypożyczenia. Powoduje to zmniejszanie ilości dostępnego sprzętu do wypożyczeń dla innych studentów.

Po zwrocie przedmiotów przez studenta wykładowca oznacza wypożyczenie jako zwrócenie. Powoduje to powrót sprzętu do tabeli i możliwość wypożyczenia go przez innych studentów.

Rozdział 5

Implementacja systemu

(tutaj brak zmian jeszcze)

5.1 Backend

Backend systemu IoTLab opiera się o środowisko Node oraz framework Express.js. System jest zdefiniowany w pliku server.js w którym zainicjowany jest obiekt app biblioteki Express.js. Powoduje to zbudowanie serwera, który pracuje na wyznaczonym adresie i porcie.

```
1 const app = express();
```

Listing 2: Zbudowanie aplikacji serwera

```
1 app.listen(PORT, async () => {
2   await connectDB();
3   console.log(`Server running on port ${PORT}`);
4 });
```

Listing 3: Uruchomienie aplikacji serwera

Serwer grupuje metody http danej ścieżki do jednego pliku dzięki wykorzystaniu ruterów Express.js. Pozwala na to łatwiejsze zarządzanie kodem serwera.

Każdy ruter posiada zostaw operacji niezbędnych do wykonywania funkcjonalności dostarczanych przez serwer. System posiada następujące rutery:

- uploads – zarządzanie plikami
- auth – logowanie użytkowników
- users – zarządzanie użytkownikami

```
app.use('/uploads', express.static('uploads'));
app.use('/auth', authRoutes);
app.use('/users', userRoutes);
app.use("/categories", categoryRoutes);
app.use('/products', productRoutes);
app.use('/orders', orderRoutes);
app.use('/notes', noteRoutes);
app.use("/materials", materialRoutes);
```

Listing 4: Routery serwera

```
const router = express.Router();
router.post("/available", checkAvailable);
router.post("/inDemand", authMiddleware, checkInDemand);
router.get("/public", getProductsPublic);
router.get("/private", authMiddleware, getProductsPrivate);
router.get("/publicId/:productId", authMiddleware, getProductByIdPublic);
router.get("/privateId/:productId", authMiddleware, getProductByIdPrivate);
router.post("/create", authMiddleware, filesUploadProduct.fields([{ name:
↪  "images", maxCount: 5 }, { name: "attachments", maxCount: 3 }]),
↪  createProduct);
router.put("/:productId", authMiddleware, filesUploadProduct.fields([{ name:
↪  "images", maxCount: 5 }, { name: "attachments", maxCount: 3 }]),
↪  updateProduct);
router.delete("/:productId", authMiddleware, hardDeleteProduct);
router.delete("/extraField/:key", authMiddleware, deleteExtraField);
```

Listing 5: Przykładowy router

Routery przed wykonaniem zleconej operacji przechodzą przez moduł autentykacji w celu ustalenia czy dana metoda http może zostać wykonana dla danego klienta.

W przypadku poprawnej autentykacji wykonywana jest odpowiednia funkcja realizująca wymaganą funkcjonalność.

```
export const getProductsPrivate = async (req, res, next) => {
  try {
    const role = req.user.role;
    const user = req.user.id;
    if (!mongoose.Types.ObjectId.isValid(user)) {
      return next(new ValidationError("Invalid user ID"));
    }
    let filter = { isDeleted: false }
    // Find all products
    if (role !== "admin") {
      filter = { isDeleted: false, $or: [{ owner: user }, { isShared: true }, { isVisible: true }] }
    }
    const products = await Product.find(filter).select('-isDeleted').sort({ name: 1 })
    .populate([{ path: "categories", select: "name" }, { path: "owner", select: "name" }]).lean();
    // Send response
    return res.status(200).json({ success: true, products });
  } catch (error) {
    return next(new ServerError({ message: error.message }));
  }
};
```

Listing 6: Pobieranie prywatnych obiektów użytkownika

5.2 Baza danych

System IoTLab posiada bazę danych MongoDB. Dostęp do bazy danych jest zrealizowany przy użyciu biblioteki mongoose. Pozwala ona na operowaniu na bazie danych tak na jak na standardowym obiekcie Java Script.

Baza danych wymaga zainicjalizowania obiektu moongose i podaniu mu danych logowania do serwera MongoDB.

```
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("Connection created successfully");
  } catch (error) {
    console.error("Connection failed:", error.message);
    process.exit(1);
  }
};
```

Listing 7: Połączenie z bazą danych

Biblioteka mongoose wymaga zaprojektowania modeli danych. Przechowują one informacje o schemacie danych przy pomocy, którego możliwe jest dokonywanie operacji na kolekcji bazy danych MongoDB.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["admin", "lecturer"], default: "lecturer",
  ↪  required: true },
  labRooms: [{ type: String }],
  isActive: { type: Boolean, default: true },
  tokenVersion: { type: Number, default: 0 },
  passwordChangedAt: Date,
  resetPasswordOtp: String,
  resetPasswordOtpExpires: Date,
});
```

Listing 8: Przykładowy model bazy danych platformy IoTLab

Modele pozwalają korzystać z wbudowanych w bibliotekę funkcji sprawdzających poprawność obiektów z modelem.

Model Obiektu posiada pole extraFields. Jest one mapą wartości typu łańcuch znaków.

Pozwala ono dodawać do obiektu dodatkowe parametry opisujące jego cechy. Rozwiązanie to jest wykorzystywane o menadżerze tabli modułu Objects.

```
extraFields: { type: Map, of: String, default: {} }
```

Listing 9: Pole extraFields

5.3 Frontend

Frontend systemu IoTLab korzysta z biblioteki React. Pozwala ona na budować własne znaczniki html. Dzięki temu budowanie stron internetowych jest szybsze i łatwiejsze.

Frontend jest podzielony na trzy części:

- App.jsx – plik startowy, który buduje całą aplikację;
- Pages – folder zawierający strony internetowe do dedykowanych ścieżek;
- Components – folder zawierający elementy wielokrotnego użytku renderowane wielokrotnie na stronach.

App.jsx jest to plik, w którym zbudowany jest komponent App.

Zawiera on definicje całej aplikacji.

Aplikacja jest zbudowana jako Router React z ustalonymi ścieżkami. Podczas próby przejścia do wybranej ścieżki router renderuje ustalony wcześniej komponent.

```
<Router>
      <Routes>
        {/* Public Routes */}
        <Route path="/" element={<WelcomePage />} />
        <Route path="/login" element={<Login />} />
        <Route path="/forgot-password" element={<PasswordResetFlow />} />
        <Route path="/unauthorized" element={<>Unauthorized</>} />
        <Route path="/redirect" element={<RootRedirect />} />
        <Route path="/logout" element={<Logout />} />
        <Route path="/cart" element={<CartPage />} />
        <Route path="/guest-products/:menuCategory" element={<GuestProducts />}
        ↪  />
        <Route path="/mainguest" element={<MainGuest />} />
```

Listing 10: Fragment aplikacji

Router składa się z publicznych i prywatnych ścieżek. Próba uzyskanie dostępu do ścieżki bez poprawnego tokenu użytkownika zostanie odrzucona.

5.4 Biblioteki dodatkowe

5.4.1 Wysyłanie wiadomości e-mail

Wysyłanie wiadomości e-mail jest realizowane przy użyciu biblioteki Nodemailer.

```
export function generateVerificationOtpEmailTemplate(otpCode) {
  return `
  <div style="font-family:Arial, sans-serif; max-width:600px; margin: 0 auto;
  ↪  padding: 20px; border-radius: 8px; background-color:#000; color:#fff;">
    <h2 style="color:#fff; text-align:center;">Verify Your Email Address</h2>
    <p style="font-size: 16px; color: #ccc;">Dear User,</p>
    <p style="font-size: 16px; color: #ccc;">To complete your registration or
    ↪  login, please use the following verification code:</p>
    <div style="text-align:center; margin:20px 0;">
      <span style="display: inline-block; font-size:24px; font-weight:bold;
      ↪  color:#000; padding:10px 20px; border: 1px solid #fff; border-radius:
      ↪  5px; background-color: #fff;">
        ${otpCode}
      </span>
    </div>
    <p style="font-size: 16px; color: #ccc;">This code is valid for 15 minutes.
    ↪  Please do not share this code with anyone.</p>
    <p style="font-size: 16px; color: #ccc;">If you did not request this email,
    ↪  please ignore it.</p>
    <footer style="margin-top: 20px; text-align: center; font-size:14px;
    ↪  color:#666;">
      <p>Thank you, <br> BookWorm Team</p>
      <p style="font-size: 12px; color: #444;">This is an automated message.
      ↪  Please do not reply to this email</p>
    </footer>
  </div>`;
}
```

Listing 11: Przykład formatowania wiadomości e-mail wysyłanej podczas resetowania hasła użytkownika

5.4.2 Pages

Strony internetowe wykorzystują API backendu w celu pobierania z niego informacji o zasobach systemu.

```
onst Login = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();
  const { login } = useAuth();
```

Listing 12: Hook'i stanu wykorzystywać przy logowaniu użytkownika do systemu

Rozdział 6

Testy jednostkowe i funkcjonalne

W ramach testów systemu przygotowano dwa podejścia testowe. Pierwsze z nich dotyczyło poprawności działania kodu systemu przy pomocy testów jednostkowych. Drugie podejście dotyczyło poprawnego działania logiki systemu i łatwości obsługi system przez użytkowników.

Testy jednostkowe zostały podzielone na dwie części. Pierwsza część miała za zadanie przetestowanie poprawności działania serwera systemu, a druga cześć frontednową. Serwer został przetestowany pod kontem poprawnych i błędnymi zapytań metod http, które miał za zadanie symulować pracę systemu.

Przeprowadzono również testy zapytań kierowanych do bazy danych. Obejmowały one zarówno poprawne i błędne zapytania.

```
1  it("should create a user", async () => {
2      const res = await request(app).post("/users/create").send({
3        name: "Jan",
4        email: "jan@example.com",
5        password: "123456",
6        role: "lecturer",
7      });
8      expect(res.statusCode).toBe(201);
9      expect(res.body.success).toBe(true);
10     const userInDb = await User.findOne({ email: "jan@example.com" });
11     expect(userInDb).not.toBeNull();
```

Listing 13: Przykładowy test jednostkowy rejestracji nowego użytkownika

Testy wykazały działanie zgodne z założeniami. W przypadku niepoprawnych danych serwer odpowiada wysyłając kody błędu informujące o niepowodzeniu wykonania żądanej operacji.

Każde miejsce w kodzie, w którym dokonywane są operacje komunikacji części frontendowej z backendem lub serwera i baza danych zostały umieszczone w blokach try/catch. Pozwala to na obsługę błędów i reagowanie na nie bez przerw w pracy serwera. W niektórych miejscach, gdzie zapytania były proste zastosowano sprawdzanie poprawności przy użyciu instrukcji *if*, w celu zwiększenia czytelności kodu.

```
useEffect(() => {
    const fetchCategories = async () => {
      try {
        const res = await
        ↪ axios.get(`${import.meta.env.VITE_API_IP}:${import.meta.env.
        ↪ VITE_API_PORT}/categories/public`);
        setCategories(res.data.categories || []);
      } catch (err) {
        console.error("Error fetching categories:", err);
      }
    };
    fetchCategories();
  }, []);
```

Listing 14: Przykładowy blok try-catch

Testy manualne były przeprowadzone przez programistów na bieżąco w czasie budowy systemu jak również przez testerów po zakończeniu prac programistów. Wykazały one poprawne działanie systemu oraz intuicyjność korzystania z systemu. Osoba mająca pierwszy kontakt z systemem nie potrzebowała szkolenia w celu wdrożenia do korzystania z systemu.

Rozdział 7

Podsumowanie i wnioski

Celem pracy było zaprojektowanie i zbudowanie systemu wspomagającego zarządzanie laboratorium dydaktycznym. System miał posiadać formę aplikacji internetowej pozwalającej na zdalny dostęp do zasobów laboratorium. Najważniejszymi zadaniami systemu było przechowywanie informacji o wyposażeniu laboratorium jak również umożliwienie wypożyczeń sprzętu i ułatwienie inwentaryzacji. Ważna była również elastyczność systemu tak aby można było przechowywać w nim różnorodny sprzet często o rożnych cechach. Równie ważne było dostosowanie go w taki sposób, aby był dostępy na urządzeniach mobilnych, bez szkody dla jego funkcjonalności.

Stan ukończenia systemu jest zgodny z założeniami początkowymi. Zaimplementowany serwer pozwala na stabilny i nieprzerwany dostęp do zasobów. Aplikacja natomiast umożliwia wygodne korzystanie z systemu zarówno na urządzeniach stacjonarnych, jak i mobilnych. System jest estetyczny i charakteryzuje się przejrzystym interfejsem. Dodatkowo jest łatwy w obsłudze oraz pozwala na promowanie sali laboratoryjnej, w której został wdrożony.

Nierelacyjna baza danych pozwala na przechowanie rożnych danych często o innych cechach. Elastyczne narzędzia wbudowane w system takie jak: drzewo kategorii, obiekty i ich cechy czy menadżer kolumn pozwalają na personalizacje systemu i jego dostosowywanie do zmieniających się potrzeb laboratorium.

System wspiera automatyzację kluczowych procesów organizacyjnych. Umożliwia wysyłkę powiadomień e-mail do użytkowników i generowaniu dokumentów, takich jak umowy wypożyczeń lub okresowe raporty, co znacznie oszczędza czas i ogranicza ludzkie błędy. Znacząco ułatwia prace opiekuna laboratorium dydaktycznego i spełnia postawione przed nim zadania. Istnieją również dalsze kierunki jego rozbudowy poprzez dodanie kolejnych modułów i funkcjonalności, które poprawią nie tylko jego użyteczność i dostępność, ale zintegrują się z istniejącą infrastrukturą uczelni. Możliwe kierunki rozwoju obejmują m.in.:

- powiadomienia SMS – umożliwiające błyskawiczne informowanie studentów oraz pracowników o zmianach w zamówieniach, awariach sprzętu lub zmiany organizacji zajęć. Dzięki temu użytkownicy nie musieliby stale monitorować systemu, ponieważ kluczowe dla nich informacje docierałby na urządzenia osobiste;
- integracje z systemami uczelnianymi – np. z Systemem Obsługi Studiów lub harmonogramami zajęć, aby ułatwić proces rezerwacji i zwrotu sprzętu. Umożliwiałoby to automatyczne powiązanie sprzętu z planem zajęć oraz umożliwić podgląd terminu zwrotu sprzętu;
- obsługę kodów QR lub technologii RFID – która umożliwi szybką identyfikację sprzętu oraz ułatwi proces inwentaryzacji. Możliwe byłoby zeskanowanie urządzenia, aby potwierdzić, że

zostało wydane lub zwrócone do sali. Proces inwentaryzacji również byłby mniej podatny na błędy;

- dedykowana aplikacja mobilna – zapewniająca pełny dostęp do zasobów systemu, również w trybie offline. Dawałoby to studentom wygodny dostęp do wszystkich funkcji systemu z poziomu urządzenia mobilnego i aplikacji zoptymalizowanej pod konkretny system;
- system rekomendacji sprzętu – sugerujący alternatywne urządzenia na podstawie historii wypożyczeń studenta lub jego dostępności. Ułatwiałoby to znajdowanie zamienników i minimalizowało ryzyko przestojów w realizacji zajęć, spowodowanego brakami sprzętu;
- możliwość zmiany języka interfejsu - co zwiększy dostępność systemu dla studentów i pracowników zagranicznych. Funkcjonalność ta bardzo dobrze wpisuje się we wsparcie programów wymiany studenckiej, jak i samą ideę umiędzynarodowienia uczelni;
- interfejs dostosowany pod osoby z niepełnosprawnościami - zapewniający zgodność z wytycznymi WCAG (Web Content Accessibility Guidelines) i ułatwiający korzystanie z aplikacji wszystkim grupom osób z niepełnosprawnościami (np. wzroku, słuchu czy ograniczeniami ruchowymi).

Platforma IoTLab stanowi alternatywę dla płatnych systemów korporacyjnych lub mało elastycznych rozwiązań dedykowanych. Dzięki dostosowaniu do urządzeń mobilnych pozwalana na łatwe korzystanie z systemu zarówno studentom jak i wykładowcą. Intuicyjny interfejs użytkownika pozwala na szybkie wdrożenie użytkowników do korzystania z systemu.

Literatura

[1] Abiona, A. A., Badmus, L. A., Ganiyu, M., & Ajinaja, M. O. (2024). Design and Implementation of a Web-Based Laboratory Management System for Efficient Resource Tracking. *Asian Journal of Electrical Sciences*, 13(2), 19–24. URL: https://doi.org/10.70112/ajes-2024.13.2.4248

[2] AddWebSolution. (2024). Benefits of Using Express.js for Web Development. *AddWebSolution Blog*. URL: https://blog.addwebsolution.com/benefits-of-using-expressjs-for-web-development

[3] Appinsnap (2024). *MEAN vs. MERN vs. LAMP for Modern Web Development*. URL: https://appinsnap.com/blogs/mean-vs-mern-vs-lamp

[4] Capella Solutions (2023, Oct 3). *MySQL vs. MongoDB: Comparing the Database Giants*. URL: https://www.capellasolutions.com/blog/mysql-vs-mongodb-comparing-the-database-giants

[5] Danaher Life Sciences. (2025, wrzesień). Laboratory Information Management System (LIMS): key features and capabilities. *Danaher Life Sciences*. URL: https://lifesciences.danaher.com/us/en/library/laboratory-information-management-system.html

[6] Deinum, M., et al. (2022). *Spring Framework Documentation*. Pivotal/VMware.

[7] de vs5003. (2024). Advantages Of Spring Boot in Java. *JavaTechOnline Blog*. URL: https://javatechonline.com/advantages-of-spring-boot-in-java/

[8] The FEBS Network (2020). *Laboratory practicals: goals, perspectives and ways of adding value to teaching labs in higher education*. URL: https://network.febs.org/posts/45724-laboratory-practicals-goals-perspectives-and-ways-of-adding-value-to-teaching-labs-in-higher-education

[9] Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.

[10] Fynd (2024). *Warehouse Management Software Cost: How Much Does It Really Cost?*. URL: https://www.fynd.com/blog/warehouse-management-software-cost

[11] GeeksforGeeks (2025, Jul 14). *MongoDB vs MySQL*. URL: https://www.geeksforgeeks.org/mongodb/mongodb-vs-mysql/

[12] Ghimire, D. (2020). *Comparative Study on Python Web Frameworks: Flask and Django*. Bachelor's Thesis, Theseus. URL: https://www.theseus.fi/handle/10024/339796

[13] Gosling, J., Joy, B., Steele, G., & Bracha, G. (2014). *The Java Language Specification*. Addison-Wesley.

[14] Grid Dynamics. (2023). Number of software developers in the world: statistics and trends. *Grid Dynamics Blog*. URL: https://www.griddynamics.com/blog/number-software-developers-world

[15] Grinberg, M. (2018). *Flask Web Development* (2nd ed.). O'Reilly Media.

[16] Hejlsberg, A., Torgersen, M., Wiltamuth, S., & Golde, P. (2010). *The C# Programming Language* (4th ed.). Addison-Wesley.

[17] Holovaty, A., & Kaplan-Moss, J. (2009). *The Definitive Guide to Django: Web Development Done Right* (2nd ed.). Apress.

[18] HQSoftware (2024). *Challenges with Outdated Legacy WMS*. URL: https://hqsoftwarelab.com/blog/challenges-outdated-legacy-wms

[19] IBM (2022). *What Is LAMP stack?* URL: https://www.ibm.com/think/topics/lamp-stack

[20] IBM (2022). *LAMP vs. MEAN: What's the Difference?* URL: https://www.ibm.com/think/topics/lamp-vs-mean

[21] Iftikhar, A., Bond, R. R., McGilligan, V., Leslie, S. J., Rjoob, K., Knoery, C., Quigg, C., Campbell, R., Boyd, K., McShane, A., & Peace, A. (2021, May 26). "Comparing Single-Page, Multipage, and Conversational Digital Forms in Health Care: Usability Study." *JMIR Human Factors*, 8(2), e25787. doi: 10.2196/25787

[22] Iguenfer, F. Z., Benhayoun, M., & Bouchaib, F. (2024). *An Overview of MEAN, MERN and LAMP Stacks: A Comparative Analysis of Architecture, Cost, Learning Curve, and Community Support*. (Preprint). URL: https://www.researchgate.net/publication/386088433_An_Overview_of_MEAN_MERN_and_LAMP_Stacks_A_Comparative_Analysis_of_Architecture_Cost_Learning_Curve_and_Community_Support

[23] inFlow (2023). *Inventory Management Software Features*. URL: https://www.inflowinventory.com/software-features/

[24] Interlake Mecalux. (2019, 30 września). ERP: definition and how it is different from a WMS. *Interlake Mecalux*. URL: https://www.interlakemecalux.com/blog/erp-definition-differences-wms

[25] Kurniawan, A. (2023). *Hallo .NET 8.0: Practical ASP.NET Core Minimal API*. Leanpub.

[26] LabLynx (2024). *LIMS vs ELN: How Do They Differ?* URL: https://www.lablynx.com/resources/articles/what-is-a-lims-eln/

[27] London School of Emerging Technology (LSET) (2025, April 4). *Why MERN Stack is the Future of Web Development*. URL: https://lset.uk/blog/why-mern-stack-is-the-future-of-web-development-learn-it-with-lset/

[28] Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

[29] Ma, X. (2016). The Research of the Opening System of University Laboratory and the Design of the Laboratory Reservation System. In *Proceedings of the 2016 International Conference on Computational Science and Engineering (ICCSE 2016)*, 113–115. Atlantis Press. URL: https://doi.org/10.2991/iccse-16.2016.19

[30] MDN Web Docs (2025). *JavaScript — MDN*. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[31] MongoDB Inc. (2023). MongoDB Community Server License. *MongoDB Legal*. URL: https://www.mongodb.com/legal/licensing/community-edition

[32] MongoDB Inc. (n.d.). *Comparing the Differences — MongoDB vs MySQL*. URL: https://www.mongodb.com/resources/compare/mongodb-mysql

[33] Mwencha, M., & Rosen, J. (2016). Better Data Visibility & Data Use Result in Lower Cost and Improved Performance in Medicine Supply Chains. In *Proceedings of the Bloomberg Data for Good Exchange Conference*, 25 September 2016, New York, USA. arXiv preprint arXiv:1609.09065

[34] NetSuite. (2025, wrzesień). What Is ERP? *NetSuite*. URL: https://www.netsuite.com/portal/resource/articles/erp/what-is-erp.shtml

[35] Novickis, L., Zarins, A., & Strazdins, G. (2021). Web Application Development Using ASP.NET Core and Microservices Architecture. *Procedia Computer Science*.

[36] Omniful (2024). *WMS Cost: Understanding Warehouse Management System Pricing*. URL: https://www.omniful.ai/blog/wms-cost-understanding-warehouse-management-system-pricing

[37] OpenJS Foundation. (2023). Node.js Licensing (MIT). *OpenJS Foundation Documentation*. URL: https://nodejs.org/en/about/licensing/

[38] Oracle (2024, May 22). *What Is the MEAN Stack? (MEAN Stack vs. MERN Stack)*. URL: https://www.oracle.com/database/mean-stack/

[39] Oracle (2024, June 26). *MEAN vs. MERN Stack Explained*. URL: https://www.oracle.com/database/mean-mern/

[40] Oracle (n.d.). *Java Platform Overview*. URL: https://www.oracle.com/java/

[41] Redmon, J. (2019). Security and Rapid Development in Django Applications. *Journal of Web Engineering*.

[42] Remotely.works (2024). *LAMP vs. MEAN/MERN: Choosing the Right Dev Stack*. URL: https://www.remotely.works/blog/lamp-vs-mean-mern-which-development-stack-is-right-for-you

[43] Ropstam (2024, Jan 1). *MERN vs LAMP – Choosing the Perfect Dev Stack*. URL: https://www.ropstam.com/mern-vs-lamp/

[44] Halton, R. (2024). *Advantages and Benefits of Using an Electronic Lab Notebook*. URL: https://www.sapiosciences.com/blog/electronic-lab-notebook-advantages-and-benefits/

[45] Zaverchand, P. (2023). 10 Reasons Why You Should Use Spring Boot. *SigmaSolve Blog*. URL: https://www.sigmasolve.com/blog/10-reasons-why-you-should-use-spring-boot

[46] Singhal, D. K. (2022). *Tech Stacks in Web Applications; Overview and Analysis*. International Journal of Scientific Research, 11(8), 143–148. URL: https://www.ijsr.net/archive/v11i8/SR22810145456.pdf

[47] Spaceo Technologies (2025, July 10). *MEAN Stack vs MERN Stack: Which Framework to Choose?* URL: https://www.spaceotechnologies.com/blog/mean-stack-vs-mern-stack/

[48] Harvard Medical School. (2025). Electronic Lab Notebooks (ELNs). *HMS Data Management*. URL: https://datamanagement.hms.harvard.edu/collect-analyze/electronic-lab-notebooks

[49] Stack Overflow Developer Survey. (2022). *Stack Overflow Developer Survey*. URL: https://survey.stackoverflow.co/2022/

[50] Staticmania. (2023). What is Spring Boot? Complete Guide. *Staticmania Blog*. URL: https://www.staticmania.com/advantages-and-disadvantages-of-spring-boot/

[51] System WMS – zarządzanie magazynem. (2025, sierpień). *DataConsult*. URL: https://dataconsult.pl/system-wms/

[52] System WMS – definicja i opis. (b.d.). *Mfiles.pl*. URL: https://mfiles.pl/pl/index.php/System_WMS

[53] TatvaSoft (2025, June 13). *MEAN Stack vs MERN Stack: Which Stack to Choose?* URL: https://www.tatvasoft.com/blog/mean-stack-vs-mern-stack/

[54] Tilkov, S., & Vinoski, S. (2010). Node.js: Using JavaScript to Build High-Performance Network Programs. *IEEE Internet Computing*, 14(6), 80–83. URL: https://doi.org/10.1109/MIC.2010.145

[55] Townsquare Interactive. (2024, sierpień 6). Top 5 Inventory Management Software Options for Small Businesses. *Townsquare Interactive Blog*. URL: https://www.townsquareinteractive.com/blog/small-business-inventory-management-software/

[56] TradeGecko (2020). *Inventory Management for Small Business*. URL: https://www.tradegecko.com/inventory-management/small-business

[57] Tulip (2025). *Rethinking the Laboratory Information Management System*. URL: https://tulip.co/blog/rethinking-laboratory-information-management-systems/

[58] van Klaveren, C., Kooiman, K., Cornelisz, I., & Meeter, M. (2019). The Higher Education Enrollment Decision: Feedback on Expected Study Success and Updating Behavior. *Journal of Research on Educational Effectiveness*, 12(1), 67–89. URL: https://doi.org/10.1080/19345747.2018.1496501

[59] van Rossum, G., & Drake, F. L. (2009). *The Python Language Reference*. Python Software Foundation.

[60] Variance InfoTech. (2023). .NET Core vs MERN Stack: A Detailed Comparison. *Variance InfoTech Blog*. URL: https://blog.varianceinfotech.com/net-core-vs-mern-stack/

[61] Variance InfoTech. (2024). .NET Core vs MERN Stack: A Detailed Comparison. *Variance InfoTech Blog*. URL: https://blog.varianceinfotech.com/net-core-vs-mern-stack/

[62] Ventory (2025). *Open Source WMS: Considerations and Trade-offs*. URL: https://www.ventory.io/blog-posts/open-source-wms-consideration-and-trade-offs

[63] Warke, J. (2023). Getting Started with Spring Boot: Advantages, Disadvantages and Use Cases. *Medium Blog*. URL: https://medium.com/@jayeshwarke011/getting-started-with-spring-boot-advantages-disadvantages-and-use-cases-497b0f04fb86

[64] Welling, L., & Thomson, L. (2016). *PHP and MySQL Web Development* (5th ed.). Pearson.

[65] Xue, Y., Jiao, X., Wang, C., Chen, H., & Aloqaily, M. (2021). Analysis and Design of University Teaching Equipment Management Information System. *Journal of Cyber Security*, 3(3), 178–185. URL: https://doi.org/10.32604/jcs.2021.016657

[66] Zandstra, M. (2017). *PHP Objects, Patterns, and Practice* (5th ed.). Apress.

[67] Zieglmeier, V., & Lehene, A. M. (2022, February 25). Designing Trustworthy User Interfaces. arXiv preprint arXiv:2202.12915.

[68] Zoho Inventory (2024). *Cloud-Based Inventory Management Software*. URL: https://www.zoho.com/inventory/



© 2025 Krzystof Piechalak, Jerzy Piersiala, Kamil Pujanek, Szymon Szron

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu LaTeX na platformie Overleaf.